



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Tesi di Laurea Magistrale in Ingegneria Informatica

*Studio di Large Language Models a
supporto della valutazione di
programmi concorrenti*

Anno Accademico 2025/2026

Relatori

Ch.mo prof. Roberto Natella

Ch.mo prof. Luigi De Simone

Correlatori

Ing. Carmine Cesarano

Ing. Luciano Pianese

Candidato

Andrea Esposito

matr. M63001650

Sommario

Lo sviluppo di **programmi di natura concorrente**, che implementano primitive di sincronizzazione e comunicazione proprie dei **Sistemi Operativi** comporta una certa complessità, che si riflette direttamente nella difficoltà della loro valutazione. I tipici test dinamici, pur effettuando una verifica diretta del comportamento dei programmi in esecuzione, non sono sempre in grado di individuare errori legati alla sincronizzazione, alla gestione delle primitive IPC o alle interazioni concorrenti tra processi e/o thread. Molte di queste problematiche dipendono infatti dall'interleaving delle operazioni e possono manifestarsi in modo **non deterministico**, rendendo difficile sia l'automazione della valutazione sia la produzione di feedback accurati e realmente utili per lo studente. Il presente studio propone dunque la realizzazione di un'infrastruttura adatta all'**analisi** di esercizi di programmazione concorrente, integrando **test dinamici**, **analisi statica** tramite *Semgrep* e regole *personalizzate* e l'impiego di **Large Language Models (LLM)**. L'approccio prevede innanzitutto la classificazione dei programmi in diverse categorie di fallimento: **compile failure**,

crash, **timeout**, **IPC leak**, **dynamic failure** e **static failure**, oltre ai casi classificati come **correct**, così da adattare le analisi successive alla natura specifica del programma. A partire da tale classificazione sono stati progettati **prompt** dedicati per l'analisi dell'output e del codice sorgente, con particolare attenzione ai problemi tipici della programmazione concorrente. Per supportare l'affidabilità o meno delle diagnosi prodotte è stato inoltre introdotto un secondo LLM con funzione di “**giudice**”, incaricato di validare semanticamente le risposte del modello principale confrontandole con la ground truth, ottenuta tramite test dinamici, analisi statica e soluzioni corrette. La sperimentazione è stata condotta su un ampio insieme di *repository*, contenenti le sottomissioni degli studenti relative a *5 esercitazioni* di laboratorio di **Sistemi Operativi**, inerenti a semafori, monitor, thread, code di messaggi e server multithread, iscritti al corso di laurea triennale in Ingegneria Informatica dell'Università Federico II di Napoli. I risultati hanno mostrato prestazioni ottimali nel riconoscimento dei fallimenti più evidenti, come: crash, timeout e IPC leak, oltre ad una buona capacità nell'individuare e descrivere difetti presenti all'interno del codice. Le principali criticità emerse hanno riguardato soprattutto l'analisi dell'output generato, la cui struttura ridotta e spesso poco informativa ha reso più difficile per il modello formulare diagnosi accurate. A tale aspetto va aggiunta la complessità nell'analizzare casistiche richiedenti sincronizzazioni articolate o protocolli di comu-

nicazione complessi, aspetti tipici degli static failure. Nel complesso, il lavoro mostra come l'integrazione tra strumenti tradizionali di testing, analisi statica e LLM possa costituire un supporto efficace sia per la **valutazione** dei programmi sia per la produzione di **feedback didattici** più dettagliati, mirati e contestualizzati.

Indice

1	Introduzione	1
1.1	Limiti dei test dinamici	2
1.2	Approccio combinato: test dinamici e analisi statica .	3
1.3	Pipeline di valutazione automatizzata	3
1.4	Estensione tramite Large Language Models	5
1.5	Obiettivi e sfide	5
1.6	Studi Correlati	6
2	Metodologia	10
2.1	Architettura basata su LLM	10
2.2	Struttura del repository di ogni esercitazione	12
2.3	Test dinamici e analisi statica: categorie di fallimento e feedback	13
2.4	Aggiornamento test dinamici e statici	14
2.5	Procedura di Test Injection	15
2.6	Architettura e funzionamento degli script di test . . .	16
2.7	Analisi completa con LLM	17
2.8	Prompt per LLM primario	19
2.9	Prompt per LLM giudice	24
2.10	Setup Sperimentale	28
2.11	Prompt di sistema	29
2.12	Provider LLM	30

2.13	Flusso di analisi completo	31
2.14	Salvataggio risultati	33
3	Risultati	34
3.1	Distribuzione globale	35
3.2	Raccolta dei commit analizzati con LLM	36
3.3	Risultati dell'analisi per ciascuna esercitazione, modello GPT-4o	38
3.3.1	Risultati esercitazione 1 - Semafori	38
3.3.2	Risultati esercitazione 2 - Monitor	46
3.3.3	Risultati esercitazione 3 - Threads	52
3.3.4	Risultati esercitazione 4 - Messaggi	58
3.3.5	Risultati esercitazione 5 - Server Multithread	65
3.4	Risultati dell'analisi replicata mediante modello GPT- OSS-20b	71
3.4.1	Differenze tra modello GPT-4o e GPT-OSS-20b	71
3.4.2	Risultati aggregati modello GPT-OSS-20b e dif- ferenze con GPT-4o	72
3.5	Analisi statica mediante Semgrep e regole <i>Ordea</i>	76
4	Conclusioni	84
4.1	Sviluppi futuri	89
	Ringraziamenti	91

Capitolo 1

Introduzione

Lo sviluppo di programmi concorrenti rappresenta una pratica tutt'altro che banale, in quanto richiede la gestione di problematiche complesse legate alla **sincronizzazione** tra processi e/o thread, alla gestione della memoria e delle risorse **IPC**, tra cui semafori, code di messaggi e memoria condivisa. È proprio in questo contesto che si inserisce il presente studio sperimentale, che si concentra sull'analisi di esercizi di **Sistemi Operativi** progettati per coprire le principali casistiche sopra citate, mediante l'utilizzo degli **LLM**, e svolti dagli studenti del corso di laurea triennale in Ingegneria Informatica dell'Università degli Studi di Napoli Federico II. In tale ambito, risulta fondamentale per gli studenti ricevere riscontri precisi e valutazione accurata delle proprie implementazioni. Allo stesso tempo, però, per i docenti il processo di progettazione degli esercizi, raccolta delle consegne, valutazione e produzione di feedback dettagliati comporta uno sforzo

significativo, soprattutto in presenza di un elevato numero di studenti. Per questo motivo, lo studio condotto dal Professore Roberto Natella [14], relatore della presente tesi, propone la realizzazione di un sistema di **valutazione automatizzata** degli esercizi, che costituisce la base metodologica per lo sviluppo del lavoro qui presentato. Il materiale prodotto nell'ambito di questa tesi è stato inoltre pubblicato in un apposito repository GitHub [2].

1.1 Limiti dei test dinamici

Data la natura dei programmi concorrenti, non è sufficiente verificarne la correttezza esclusivamente tramite **test dinamici**. È infatti necessario analizzare anche il corretto utilizzo delle risorse a basso livello del sistema operativo. In particolare, i bug di concorrenza, come *race condition* e *deadlock*, derivanti da un uso errato o mancante delle primitive di sincronizzazione, risultano difficili da individuare durante l'esecuzione del programma, poiché si manifestano solo in rare combinazioni di eventi e condizioni di **scheduling** della CPU [5]. Analogamente, i problemi legati alla gestione delle risorse possono generare effetti collaterali che non emergono facilmente osservando il comportamento del programma. Inoltre, i test case tradizionali non sono in grado di fornire indicazioni puntuali sulla porzione di codice responsabile dell'errore.

1.2 Approccio combinato: test dinamici e analisi statica

L'articolo da cui prende avvio questa tesi affronta tali limitazioni proponendo un approccio più approfondito alla valutazione degli esercizi [14]. In particolare, vengono combinati **test dinamici**, utili a verificare la correttezza dell'output prodotto, con un'**analisi statica** mirata, basata su regole costruite *ad hoc* per ciascun esercizio. Questo consente di individuare in modo più efficace errori tipici dei programmi di sistema, legati soprattutto alla sincronizzazione e alla gestione delle risorse.

1.3 Pipeline di valutazione automatizzata

Il sistema di partenza prevede una **pipeline automatizzata** per la raccolta, l'analisi delle sottomissioni e la distribuzione del feedback, sfruttando strumenti come **GitHub Classroom** [6]. Gli esercizi vengono distribuiti agli studenti sotto forma di repository contenenti codice template, arricchito da commenti “TBD” (to be done), che indicano le parti da completare, e dalla traccia dell'esercizio all'interno di un file README. Ogni volta che uno studente effettua un commit, il processo di valutazione viene avviato automaticamente e si articola in tre fasi principali:

1. **Compilazione**

2. Esecuzione

3. Analisi statica

Nel caso in cui il codice non superi una di queste fasi, il processo viene interrotto e viene generato un messaggio di **feedback**, pubblicato ad esempio nella sezione delle *issues* del repository su GitHub. Tale messaggio riporta l'esito della valutazione e, nel caso dell'analisi statica, indica anche la posizione del codice considerata errata. Lo studente ha quindi la possibilità di correggere il programma e inviare una nuova versione, rendendo il processo iterativo e garantendo un feedback rapido e continuo.

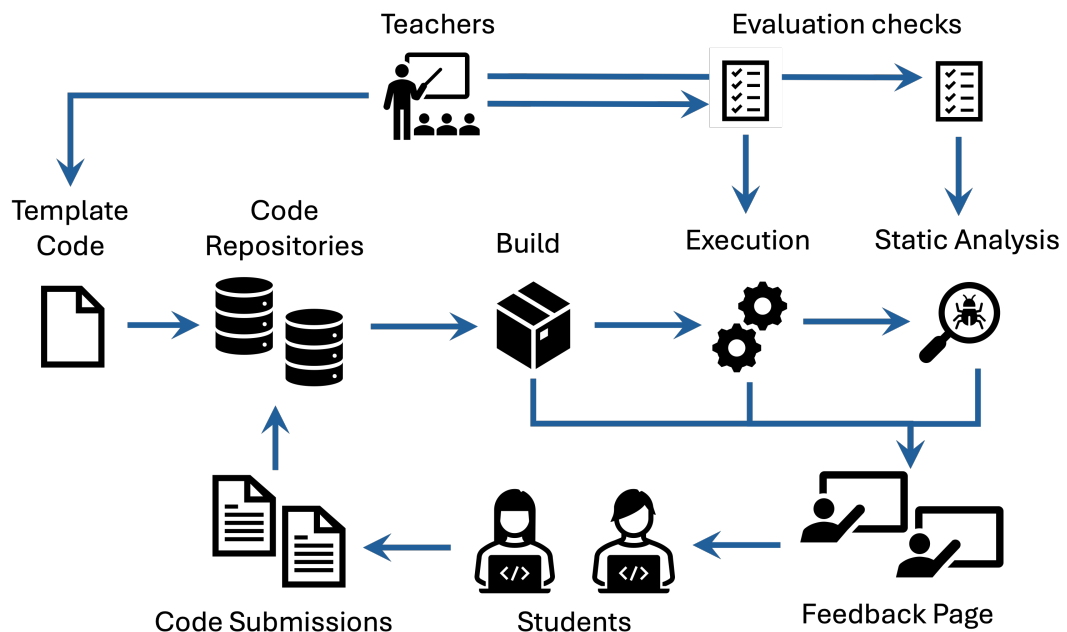


Figura 1.1: Pipeline di valutazione automatizzata degli esercizi [14]

1.4 Estensione tramite Large Language Models

A partire da questa base, il lavoro sviluppato in questa tesi si propone di estendere il sistema esistente attraverso l'utilizzo dei **Large Language Models (LLM)**, con l'obiettivo di supportare ulteriormente la valutazione dei programmi concorrenti. Sebbene l'uso di strumenti di analisi statica e regole definite ad hoc risulti già efficace, come evidenziato anche nei risultati dello studio di riferimento [14], l'impiego di modelli di intelligenza artificiale introduce nuove possibilità, consentendo di generare **feedback più flessibili e contestualizzati** rispetto a quelli prodotti da script deterministici.

1.5 Obiettivi e sfide

In un contesto in cui gli LLM stanno assumendo un ruolo sempre più centrale in diversi ambiti applicativi, appare quindi interessante valutare la loro capacità di analizzare ed individuare anomalie in programmi di natura concorrente, tenendo conto delle specificità legate alla gestione della concorrenza e delle risorse. In particolare, l'obiettivo è quello di verificare se tali modelli siano in grado di affiancare l'analisi statica tradizionale, contribuendo sia all'identificazione di errori nel codice sia all'analisi dell'output prodotto dai programmi, come nel caso dei messaggi scambiati tra processi. Una delle principali sfide

affrontate riguarda la definizione di **prompt adeguati** da fornire agli LLM, che risultino sufficientemente generici da poter essere applicati a esercizi differenti, pur mantenendo un buon livello di accuratezza. L'obiettivo è stato quindi costruire un sistema robusto e adattabile, in grado di integrare diverse tecniche di analisi.

1.6 Studi Correlati

L'analisi di programmi tramite l'utilizzo di **Large Language Models (LLM)**, in combinazione con **test deterministici** e strumenti tradizionali di **analisi statica** come **Semgrep**, **CodeQL** e simili, rappresenta un ambito di crescente interesse. Questo sia per confrontarne le capacità e le differenze, sia per individuare problematiche che difficilmente emergerebbero attraverso regole statiche generiche, a meno di non definirle in modo altamente personalizzato [14], operazione che però richiede uno sforzo non trascurabile. Inoltre, applicare queste tecniche su larga scala, ad esempio nel contesto di corsi universitari con numerosi studenti, consente di raccogliere un'ampia varietà di soluzioni e di supportare gli studenti nel migliorare le proprie competenze attraverso **feedback mirati**. In questo contesto si inserisce un particolare studio, che si focalizza sull'utilizzo degli LLM per generare **feedback** con l'obiettivo di ridurre il divario rispetto a quello prodotto da script di test, per gli studenti dei corsi di programmazione [3]. In particolare, il lavoro prevede la raccolta degli esercizi svolti dagli studenti di

un corso introduttivo a Python tramite una piattaforma chiamata **Gippy**, adottando un approccio per certi versi simile a quello considerato nella presente tesi. I feedback vengono generati dagli LLM attraverso **prompt specifici**. I risultati evidenziano tuttavia alcune criticità, in quanto i feedback prodotti dal modello **GPT-3.5-turbo** risultano spesso imprecisi, mentre si osserva un miglioramento significativo con l'utilizzo di **GPT-4**. Un approccio ulteriormente orientato all'impiego degli LLM in ambito didattico si pone come obiettivo quello di fornire agli studenti un supporto immediato e utile, senza però esporre direttamente soluzioni complete del codice [8]. A tal fine è stato sviluppato un assistente, denominato **CodeAid**, che genera *pseudocodice* accompagnato da spiegazioni dettagliate e annota il codice dello studente evidenziando gli errori e suggerendo possibili correzioni. Lo studio ha coinvolto anche otto docenti di programmazione, al fine di raccogliere valutazioni sull'efficacia dello strumento. L'applicazione è avvenuta in un corso di C e sistemi operativi, quindi in un contesto molto vicino a quello di questa tesi. Tuttavia, a differenza dell'approccio qui proposto, non vengono utilizzati test deterministici né strumenti di analisi statica, ma si fa affidamento esclusivamente sugli LLM come assistenti, sebbene attraverso l'uso di **prompt accuratamente progettati**. Un'impostazione simile è adottata anche in un interessante studio, in cui è stato sviluppato un assistente basato su LLM, chiamato **CodeTutor** [10]. Questo strumento è stato impiegato per analizzare

e fornire feedback agli studenti di un corso introduttivo di programmazione in Python, con l'obiettivo di valutare l'efficacia degli LLM nel migliorare le capacità degli studenti e, al tempo stesso, evidenziare la necessità di definire metodologie adeguate per massimizzare i benefici in ambito educativo. Gli studi discussi finora non prevedono l'utilizzo combinato di strumenti di analisi statica. In questa direzione risulta particolarmente interessante uno speciale lavoro, che evidenzia come l'analisi statica, pur essendo fondamentale per l'individuazione dei bug, possa risultare limitata quando applicata a codice complesso [9]. Per superare questo limite, viene proposta un'integrazione con gli LLM attraverso un framework denominato **LLift**. L'approccio si concentra in particolare sull'individuazione di bug di tipo “*Use Before Initialization*” (*UBI*) all'interno del kernel Linux, riuscendo ad identificare quattro bug UBI precedentemente sconosciuti nel kernel Linux mainstream. Questo lavoro è particolarmente rilevante perché si colloca anch'esso nel contesto dei **sistemi operativi** e fa uso di **prompt specifici** per interrogare gli LLM, sebbene non sia orientato alla valutazione di esercizi svolti da studenti. Un ulteriore contributo in questa direzione è rappresentato da uno studio, in cui viene proposta un'integrazione più stretta tra **analisi statica** e LLM [1]. In questo caso, i risultati dell'analisi statica, ottenuti tramite l'analizzatore **EESI**, vengono utilizzati per costruire i prompt da fornire all'LLM. A loro volta, le risposte del modello vengono sfruttate per guidare le fasi successive

dell'analisi. L'approccio è stato applicato a codice scritto in linguaggio C, mostrando come le due tecniche possano rafforzarsi a vicenda se opportunamente combinate. Tra gli agenti più famosi ed utilizzati in questo ambito vi è senza dubbio **ChatGPT**, con i suoi diversi modelli disponibili, utilizzato anche in diversi studi recenti. In particolare, è stato analizzato un particolare lavoro legato alla valutazione dell'efficacia di ChatGPT in attività di **analisi statica**, concentrandosi sul rilevamento di bug quali “*Null Dereference*” e “*Resource Leak*”, nonché sulla riduzione dei falsi positivi [13]. In questo studio viene utilizzato l'analizzatore statico **Infer** per raccogliere esempi di tali bug da dieci progetti open-source. Anche in questo caso si fa uso di **prompt template**, analogamente a quanto avviene nella presente tesi. Tuttavia, l'obiettivo principale è migliorare i risultati dell'analisi statica tradizionale, in presenza di possibili falsi positivi, mentre nel lavoro qui presentato l'analisi statica si basa su regole altamente personalizzate e quindi più affidabili, venendo affiancata dagli LLM in ottica **complementare** piuttosto che correttiva.

Capitolo 2

Metodologia

2.1 Architettura basata su LLM

L’infrastruttura definitiva combina, da un lato, test dinamici e regole di analisi statica, e dall’altro l’utilizzo di **prompt specifici** per l’analisi del codice e dell’output tramite LLM. La pipeline risultante consente quindi una valutazione più completa dei programmi sviluppati dagli studenti, fornendo sia feedback deterministici, derivanti dalle tecniche tradizionali, sia feedback generati dai modelli di intelligenza artificiale. Particolare attenzione è stata inoltre dedicata alle limitazioni degli LLM, che possono soffrire di fenomeni di *hallucination*. Per mitigare questo problema, si è scelto di introdurre un ulteriore livello di validazione basato su un LLM “**giudice**”, incaricato di verificare la correttezza delle diagnosi prodotte dal modello principale. A tal fine, vengono utilizzati prompt dedicati, ai quali vengono forniti come rife-

rimento i risultati dei test dinamici (considerati come *ground truth* per l'analisi dell'output) e i warning derivanti dall'analisi statica insieme al codice della soluzione (come *ground truth* per l'analisi del codice). In questo modo è stato possibile realizzare un sistema completo per l'analisi automatizzata degli esercizi svolti dagli studenti, la cui architettura complessiva è illustrata nella Figura 2.1.

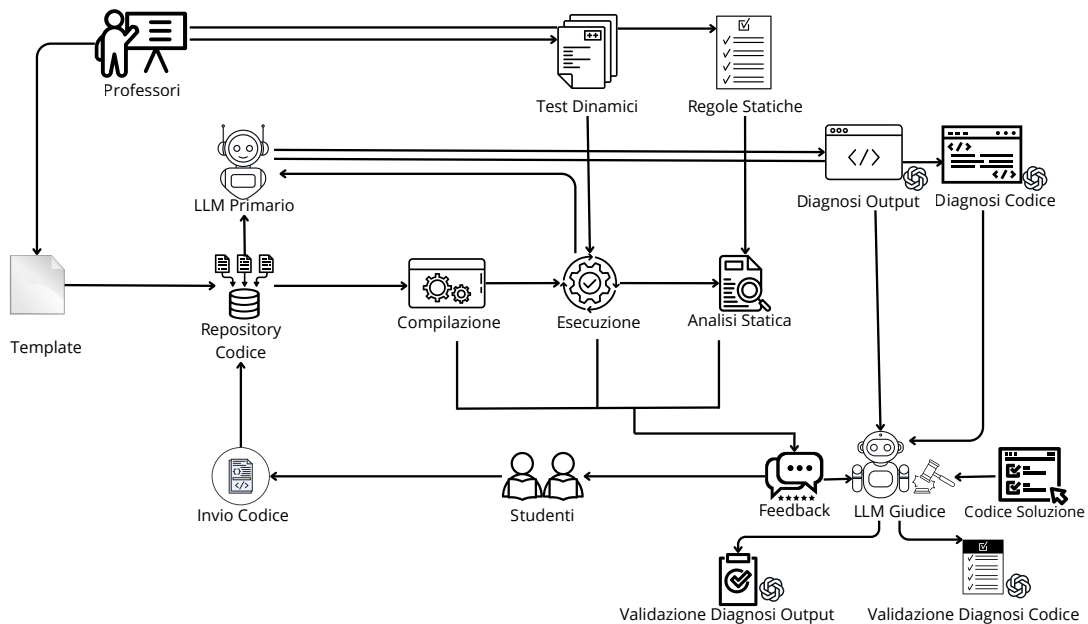


Figura 2.1: Pipeline di analisi con LLM

Il **dataset** di partenza è stato raccolto dalla classe dedicata su **GitHub Classroom** relativa al corso di *Sistemi Operativi* del 2023, seguito dagli studenti del corso di laurea triennale in Ingegneria Informatica dell'Università di Napoli Federico II. La classe comprende cinque esercitazioni: 1-*Semafori*, 2-*Monitor*, 3-*Threads*, 4-*Messaggi*, 5-*Server Multithread*, ciascuna contenente repositories studenti.

2.2 Struttura del repository di ogni esercitazione

La struttura di ogni repository studente, all'interno della directory relativa alla specifica esercitazione, comprende:

- una cartella denominata `".test"`, la quale contiene 3 file, denominati: *preprocessor.pl*, *test.sh*, *so-highlight.config*.
- una cartella dedicata per ogni esercizio, che contiene:
 - il codice sorgente (.c) sviluppato dallo studente a partire dal template fornito, completando i blocchi contrassegnati dai commenti *TBD* (to be done);
 - i relativi file di libreria (.h);
 - il Makefile per la compilazione;
 - il file README.md contenente la traccia dell'esercizio;
 - un'ulteriore cartella denominata `".test"` contenente script bash specifici per il test dell'output generato dal programma, che ereditano le funzioni di analisi (compilazione, analisi output, analisi statica) dallo script `"test.sh"` presente nella cartella esterna `".test"`;
 - file in formato *YML* contenenti le regole di analisi statica *semgrep* costruite ad hoc per il relativo esercizio.

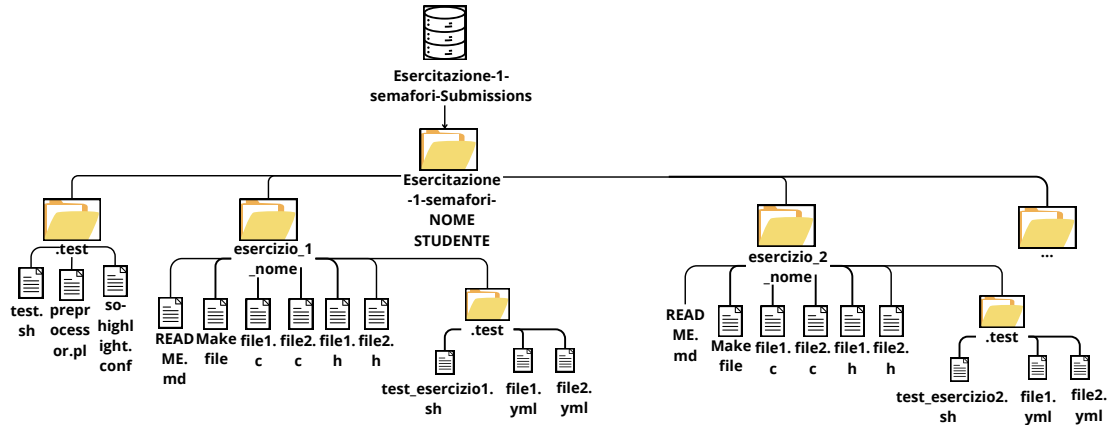


Figura 2.2: Struttura repository per esercitazione

2.3 Test dinamici e analisi statica: categorie di fallimento e feedback

I test dinamici e le regole di analisi statica sono stati sviluppati nello studio originale [14], ma presentavano alcune debolezze che sono state affrontate e risolte nel presente lavoro. Un'esigenza immediata è stata quella di classificare ciascun programma analizzato in base alla relativa **categoria di fallimento**, così da sfruttare pienamente l'esito dei test e guidare le successive analisi sul programma in funzione della categoria assegnata. È stato tuttavia necessario migliorare la capacità di individuare correttamente la categoria di fallimento di ciascun programma, introducendo meccanismi di verifica più accurati. In questo modo è stato possibile definire sei categorie di fallimento, oltre ai programmi classificati come **correct**, ossia quelli che superano sia i controlli dinamici sia quelli statici. Le categorie, in ordine di fallimen-

to, sono: **compile failure**, **crash**, **timeout**, **IPC leak**, **dynamic failure**, **static failure**. Sono state inoltre aggiunte diagnostiche più dettagliate per ogni esercizio, all'interno dei relativi script di test, pensate come feedback dell'analisi destinato agli studenti, evidenziando più precisamente il particolare errore riscontrato durante l'esecuzione del programma. Il cosiddetto **test feedback** fornisce così un messaggio preciso relativo al problema individuato all'interno dell'output, iterando sulle linee prodotte in esecuzione e analizzando i messaggi scambiati tra processi, i valori calcolati e altri elementi rilevanti. In caso di **fallimento statico**, invece, viene generato un messaggio che indica direttamente il nome del file sorgente contenente gli errori. Allo stesso modo i **file YML** presenti all'interno della medesima cartella di test contengono regole di analisi statica *semgrep* nella forma: **id** (contenente il nome della regola), **message** (specifico warning da segnalare allo studente), **patterns** (snippet di codice che la regola cerca), **pattern-not** (snippet di codice errato che genererebbe il warning statico).

2.4 Aggiornamento test dinamici e statici

Si è reso necessario aggiornare alcune delle **regole statiche** che generavano **falsi positivi**, intervenendo in alcuni casi eliminando regole fuorvianti per l'esercizio in questione, ed in altri casi modificandole attraverso una revisione dei pattern da escludere o meno. Anche i **test**

dinamici sono stati ulteriormente revisionati, data la generazione di falsi negativi dovuti principalmente all'interleaving dei messaggi e alla severità eccessiva dei controlli sull'output. Le modifiche introdotte hanno mantenuto coerenza con le versioni originali dei test, ma hanno permesso di ridurre significativamente tali casi, con un conseguente aumento dei programmi classificati come **correct** e **static failure**.

2.5 Procedura di Test Injection

Dato l'aggiornamento dei test dinamici e statici per le diverse esercitazioni, si è reso necessario introdurre un processo di **test injection** antecedente all'analisi. Questo passaggio consente di sovrascrivere i test obsoleti con le versioni aggiornate per ciascun commit da analizzare, garantendo così coerenza e affidabilità nei risultati. Il processo viene inglobato e diviene parte dell'analisi automatizzata condotta, sovrascrivendo le cartelle **".test"** con i relativi test dinamici e regole statiche dell'esercizio oltre alla cartella comune a tutti gli esercizi. Questo approccio consente di applicare modifiche e aggiornamenti ai test in qualsiasi momento, senza la necessità di intervenire manualmente su ogni repository: l'aggiornamento viene infatti propagato automaticamente a tutti i commit analizzati.

2.6 Architettura e funzionamento degli script di test

Particolare importanza rivestono gli script di test sorgenti denominati "**test.sh**", che implementano le principali funzioni di testing, ereditate dagli script di test specifici dei singoli esercizi, che a loro volta integrano la logica necessaria per verificare il comportamento richiesto da ciascun esercizio. Il passaggio iniziale consiste nell'inizializzare il feedback finale, che verrà salvato in un file denominato */tmp/feedback.md* con modalità *append*, all'interno della repository dello studente. Successivamente avviene la pulizia delle **risorse IPC** residue da esecuzioni precedenti, in modo che non interferiscano con il test. Viene così avviato il **processo di testing** attraverso tre fasi: **compilazione** tramite comando *gcc*, **esecuzione** e salvataggio dell'output nel file */tmp/output.txt*, **analisi statica** con *semgrep* e salvataggio dei warnings in formato *JSON* (file, line, message). Infine viene controllato che tutte le risorse IPC siano state deallocate. Una funzione, denominata **failure**, viene invocata ogni qual volta si verifica un fallimento durante il processo di testing, generando rispettivamente un feedback specifico a seconda del tipo di fallimento, in ordine: **compile failure** (la compilazione è fallita), **crash** (il programma è andato in crash), **timeout** (il programma è andato in timeout), **IPC leak** (è necessario deallocare le risorse IPC), **dynamic failure** (l'output fina-

le non corrisponde a quello richiesto), **static failure** (l'analisi statica effettuata con `semgrep`, utilizzando regole personalizzate, ha generato warning). Per quanto riguarda i fallimenti dinamici vengono inoltre generati messaggi dettagliati, che completano il feedback finale a seconda dello specifico errore. Tale operazione viene effettuata sfruttando gli **script Perl** di test dello specifico esercizio in esame. Come descritto in precedenza quest'ultimi includono il sorgente "`test.sh`", evitando di dover replicare le procedure di compilazione, esecuzione, analisi statica e gestione del feedback, ma di concentrarsi solo sulla valutazione della logica specifica dell'esercizio.

2.7 Analisi completa con LLM

A partire da tale infrastruttura sono state introdotte analisi basate su LLM, con l'obiettivo di integrare e completare le verifiche precedenti, ottenendo così risultati più completi. Tali analisi sono state condotte attraverso **prompt template**, in **lingua Inglese** per maggiore qualità, adattabili a qualunque esercizio ed in grado di fornire diagnosi quanto più accurate possibile. In particolare viene separata l'analisi dell'output del programma da quella del relativo codice implementato dallo studente e viene richiesta sia una classificazione binaria output/codice corretto (YES/NO) che una diagnosi testuale. Il processo di creazione dei prompt ha seguito le pratiche tipiche del **prompt engineering**, facendo uso di specifiche **decision rules** e di una serie di

indicazioni mirate a guidare il comportamento del modello. Questo ha permesso di strutturare richieste coerenti, controllabili e capaci di produrre analisi affidabili. Ragionando in merito alle analisi necessarie si è reso opportuno introdurre un ulteriore LLM, distinto da quello considerato **primario**, il quale è incaricato di diagnosticare output e codice di un programma, ma producendo talvolta diagnosi errate o fuorvianti a causa dei tipici fenomeni di **hallucination**, distanti dal reale problema del programma. Tale LLM è stato soprannominato **giudice**, poiché, a differenza del modello primario, dispone della **ground truth**: conosce la reale causa del problema ed ha accesso al codice della soluzione corretta. Sfruttando queste informazioni, il giudice valida le diagnosi prodotte dal modello principale, identificando eventuali errori o interpretazioni imprecise. In questo modo è possibile ottenere risultati affidabili pur facendo uso di strumenti meno deterministici rispetto agli script di test tradizionali, come gli LLM. L'integrazione del modello "giudice" consente infatti di verificare che le valutazioni siano coerenti con la reale natura del problema. Avendo realizzato la classificazione delle categorie di fallimento dei programmi, si è ritenuto opportuno che ciascun caso venisse analizzato dagli LLM in modo differente, in funzione della propria categoria di appartenenza. Quest'approccio ha previsto dunque la definizione di diversi prompt, permettendo ai modelli di adottare criteri di valutazione più mirati e pertinenti rispetto alla natura specifica dell'errore.

2.8 Prompt per LLM primario

prompt_output

Per analisi dell'output generato da programmi di categorie *dynamic failure*, *static failure* e *correct*, riceve: **traccia** dell'esercizio e **output** del programma e produce classificazione binaria per l'output corretto o meno (YES/NO) e diagnosi sull'output osservato. E' costituito da 6 macroblocchi:

1. Target della valutazione: output prodotto dall'esecuzione del programma.
2. Controlli da effettuare: identità degli interlocutori (ad es. produttore-consumatore), messaggi scambiati, valori calcolati, conteggi, accoppiamenti tra messaggi, ordine dei messaggi, completamento dell'esecuzione.
3. Regole specifiche per ottenere diagnosi accurata, per un totale di 11 regole, tra cui:
 - *"In concurrent or multi-process traces, stdout interleaving by itself is not a protocol violation ONLY if each visible actor stream (per PID/role) is internally coherent and replies unambiguously reference their senders."*
 - *"If Output_Correct = NO, identify the specific broken interaction: missing trace, wrong values, wrong counts, wrong pairing, wrong ordering, or unverifiable value flows."*

4. Contesto: traccia dell'esercizio e output da analizzare.
5. Regole dettagliate per casi specifici di programmi concorrenti, per un totale di 32 regole, tra cui:
 - *"CONCURRENT LOG ORDER: In concurrent or multi-process/multi-thread systems, a server or worker may print its log entry after the client has already received and printed the reply. This is normal and does not constitute an ordering violation. Evaluate protocol correctness based on the logical sequence of operations (request sent, reply received), not on the physical order of log lines in stdout."*
6. Formato di risposta richiesto:
 - Output_Correct: YES or NO
 - Output_Diagnosis: diagnosi dettagliata, al più 100 parole.

prompt_codice

Per analisi del codice dei programmi di categorie *crash*, *timeout*, *ipc leak* e *dynamic failure*, riceve: **traccia** dell'esercizio, **output** del programma, **codice** implementato dallo **studente** e **diagnosi LLM primario** per il corrispondente **output**, producendo classificazione binaria per codice corretto o meno (YES/NO) e diagnosi sul codice fornito. Anche in questo caso la struttura è composta da 6 macroblocchi:

1. Target della valutazione: codice implementato dallo studente.

2. Controlli da effettuare, tra cui l'uso corretto dei protocolli di comunicazione, il rispetto delle policy di sincronizzazione e l'impiego appropriato delle primitive.
3. Runtime focus: viene richiesta l'individuazione della causa, all'interno del codice, che determina l'errata esecuzione del programma. L'analisi tiene conto della diagnosi fornita dall'LLM sull'output, evitando di ricercare bug generici e concentrandosi invece sul problema specifico che ha generato il malfunzionamento.
 - *"If output_diagnosis explicitly names a missing phase, wrong value, or protocol mismatch, prioritize finding the code statement that causes that specific symptom."*
4. Regole dettagliate per la programmazione concorrente, per un totale di oltre 30 regole, tra cui indicazioni per evitare di segnalare errori legati all'**error handling**, che non vengono diagnosticati dai check statici.
 - *"Do NOT diagnose error-handling defects unless they directly cause the observed runtime failure"*
 - *"Do not claim a race condition unless unsynchronized concurrent accesses to the same shared state are explicitly visible in the code."*

5. Richiesta esplicita di citare il problema rilevato con precisione, includendo: **nome del file**, **funzioni**, **API**, **variabili** e **linee di codice** quando visibili.

- "*Prefer references such as 'FILE: server.c', 'function worker', 'pthread_create(...)', 'msgsnd(...)', 'msgrcv(...)', or a short quoted code fragment with line numbers when available.'*"

6. Formato di risposta richiesto:

- Code_Correct: YES or NO
- Code_Diagnosis: diagnosi dettagliata, al più 100 parole.

prompt_codice_static

Per analisi del codice dei programmi di categoria *static failure*, riceve: **traccia** dell'esercizio e **codice** implementato dallo **studente**, producendo classificazione binaria per codice corretto o meno (YES/NO) e diagnosi sul codice osservato. La struttura segue quanto illustrato in *prompt_codice*, ma si concentra principalmente sull'**analisi strutturale** del programma. Vengono nuovamente esclusi i problemi legati all'*error handling* e l'attenzione è rivolta ai difetti strutturali che compromettono la corretta natura del codice.

- "*ERROR HANDLING RULE: Do NOT diagnose missing error-handling code as a structural defect*"
- "*Look for one structural defect using:*

- *lifetime/ownership*
- *synchronization policy*
- *scope/order of operations*
- *selector or routing logic*
- *forbidden primitive*
- *visible absence of a required mechanism in the shown code path"*

prompt_codice_correct

Per analisi del codice dei programmi di categoria *correct*, riceve: **traccia** dell'esercizio, **output** del programma e **codice** implementato dallo **studente**, producendo classificazione binaria per codice corretto o meno (YES/NO) e diagnosi sul codice osservato. La struttura segue nuovamente quanto illustrato in *prompt_codice*, ma adotta una strategia più conservativa: il modello segnala codice errato solo in presenza di bug chiaramente individuabili, privilegiando un'analisi meno sospettosa nei confronti dell'accuratezza complessiva del programma. Questo approccio riduce i falsi positivi e concentra l'attenzione sui difetti realmente evidenti.

- *"Be very conservative: a passing execution is strong evidence for YES unless the code itself shows an explicit, undeniable violation."*

2.9 Prompt per LLM giudice

prompt_giudice_output

Per la validazione della diagnosi output del LLM primario, riceve: **diagnosi** LLM sull'**output** e **feedback** dello script di **test**, e produce classificazione binaria per diagnosi output corretta o meno (YES/-NO) e motivazione della propria sentenza. Lo stile del prompt segue quanto già adottato per LLM primario, con la differenza che la presenza della **ground truth** consente una struttura significativamente più compatta. Il prompt è organizzato in diverse sezioni essenziali, mirate a guidare il modello nella validazione delle diagnosi:

- Contesto: diagnosi output LLM primario e ground truth (feedback test dinamico)
- Regole specifiche che guidano la valutazione, tra cui:
 - *"Treat detailed diagnoses as enrichments of generic ground truth. If ground truth says "execution incorrect" but does not detail why, a diagnosis identifying the specific type of error (wrong value, missing interaction, wrong pairing, wrong count) is acceptable."*
 - *"Accept the diagnosis if it correctly identifies a type of output problem that matches the ground truth failure."*
- Formato di risposta standard:

- Judge_Output_Correct: la diagnosi fornita da LLM primario rispetto all'output è corretta? (YES or NO)
- Judge_Output_Motivation: motivazione della decisione del giudice, al più 100 parole.

prompt_giudice_codice

Per la validazione della diagnosi codice del LLM primario per i casi di *static failure*, dunque con la presenza di **warning** generati dall'analisi pregressa mediante *semgrep*. Riceve: **diagnosi** LLM primario rispetto al **codice** implementato dallo **studente**, **warnings**, **snippet** rilevanti di **codice** dello **studente**, **codice** della **soluzione**, e produce classificazione binaria per diagnosi codice corretta o meno (YES/NO) e motivazione della propria sentenza. In questo caso è importante definire il modo in cui gli *snippet* di codice studente vengono raccolti: per ogni warning, a partire dal particolare file e relativo numero di linea segnalato, vengono raccolte porzioni di codice in un intorno di ± 10 linee di codice rispetto a quella del warning, così da ridurre la dimensione del prompt da inviare al "*giudice*" e verificare esattamente tali parti incriminate rispetto alla corrispondente soluzione, fornendo per quest'ultima l'intero codice sorgente corretto, ma focalizzandosi solo sulla parte da confrontare.

- "*A diagnosis identifying a root cause or prerequisite condition for the warned defect should be accepted if causally related.*"

- *"Accept the diagnosis if it identifies the same defect mechanism, even if phrased differently or at a different architectural level."*

prompt_giudice_codice_diff

Per la validazione della diagnosi codice del LLM primario per i casi di *crash*, *timeout*, *IPC leak*, *dynamic failure* e *correct*, riceve: **diagnosi** LLM sul **codice** implementato dallo **studente**, **feedback** dello script di **test** e **diff** compatto tra codice studente e codice soluzione, e produce classificazione binaria per diagnosi codice corretta o meno (YES/NO) e motivazione della propria sentenza. Come si nota, in questo caso non sono presenti warnings, dato che essi vengono generati solo per programmi classificati come *static failure*. Per questo motivo la valutazione viene effettuata utilizzando come ground truth l'esito dei test, rappresentato dal messaggio di feedback finale, ma soprattutto mediante la costruzione di un *diff* tra codice studente e codice soluzione. È evidente che differenze limitate ai nomi delle variabili, o comunque minimali, non debbano essere considerate problematiche nel confronto con la soluzione. Per questo motivo, il prompt viene esplicitamente guidato a non segnalarle, evitando così falsi positivi e concentrandosi esclusivamente su difetti che incidono realmente sul comportamento del programma.

- *"Accept only if the diagnosis matches a concrete structural difference visible in the diff and consistent with the observed failure."*

- *"Do not reject the diagnosis if the code is functionally correct and the differences in the diff are superficial (variable names, loop order, formatting) and do not constitute a correctness defect."*

Costruzione diff compatto

A partire dalla diagnosi codice generata da LLM primario vengono estratti *identificatori* (es. mutex, lock, queue) e soprattutto **nomi dei file menzionati**. Quest'ultimi vengono ordinati per rilevanza, in base a quelli menzionati per primi. Il terzo passo è quello di trovare file modificati, ovvero diversi rispetto al codice della soluzione, e dunque ordinare per priorità tutti i file raccolti (menzionati + modificati). Per ogni file modificato vengono costruiti blocchi di differenze, utilizzando *difflib.SequenceMatcher* per trovare differenze tra studente e soluzione, ed estratte ± 3 linee di contesto attorno alla differenza.

Esempio diff calcolato:

```
FILE: Server.c
STUDENT LINES 38-42:
 38: void increment() {
 39:     shared_counter++;
 40: }
 41:
SOLUTION LINES 38-44:
 38: void increment() {
 39:     pthread_mutex_lock(&lock);
 40:     shared_counter++;
 41:     pthread_mutex_unlock(&lock);
 42: }
 43:

FILE: Server.c
STUDENT LINES 55-58:
 55: void main() {
```

```
56:      pthread_create(...);  
SOLUTION LINES 55-60:  
55: void main() {  
56:      pthread_mutex_init(&lock, NULL);  
57:      pthread_create(...);
```

2.10 Setup Sperimentale

Per supportare l'intero processo di **analisi automatizzata** è stata realizzata un'**infrastruttura software** composta da un **framework** sviluppato in linguaggio *Python* e da una serie di *script bash*. Tale infrastruttura si occupa dell'esecuzione dei **test dinamici**, dell'**analisi statica**, della classificazione delle **categorie di fallimento**, dell'interazione con gli **LLM** e del salvataggio incrementale dei risultati prodotti. L'esecuzione delle analisi richiede l'utilizzo coordinato di diversi **strumenti software**, tra cui compilatori, utility Linux, *Semgrep* e *script* di testing. Per questo motivo è stato adottato **WSL** (*Windows Subsystem for Linux*), così da disporre di un ambiente Linux coerente con quello previsto dai test e compatibile con tutti i tool necessari. In particolare, **WSL** ha consentito di eseguire gli *script bash*, installare le dipendenze richieste e automatizzare l'intera **pipeline di analisi**, mantenendo uniformità tra **ambiente di sviluppo** ed **ambiente di esecuzione**.

2.11 Prompt di sistema

Con lo scopo di migliorare ulteriormente le performance degli LLM sono stati definiti due prompt, rispettivamente per LLM primario e giudice, definiti **System prompt**, che contengono importanti indicazioni e che vengono aggiunti in cima ad ogni prompt inviato rispettivamente ad ognuno dei due LLM.

Primary System prompt

```
You analyze student C solutions for Operating Systems exercises.
Use only the provided evidence. Do not invent hidden requirements,
hidden expected values, invisible code paths, or unsupported causes
. Prefer conservative, evidence-based judgments. Do not overclaim
correctness from partial traces, later side effects, or loosely
related code details. Keep the boolean field and diagnosis
consistent. Write diagnoses in English, using at most 100 words.
Return exactly the requested fields. DO NOT use markdown formatting
, backticks, or code blocks. Return plain text only.
```

Judge System prompt

```
You are a strict semantic grader. Accept only diagnoses that match
the same concrete symptom or defect as the ground truth. Reject
generic, speculative, partially related, unsupported, or different
diagnoses. When the ground truth is generic, do not promote it into
a more specific diagnosis. When the ground truth says the
submission is correct, accept concise no-defect diagnoses unless
they invent a problem. Return exactly the requested field.
```

Infine è previsto un ulteriore prompt detto di *self refinement*, il quale riceve la diagnosi fornita dal LLM primario e la revisiona, così da produrre una valutazione finale più precisa rispetto a quella iniziale. Uno dei compiti di tale prompt è garantire consistenza tra classificazione binaria (YES/NO) e rispettiva diagnosi, laddove non fosse così.

Il processo di ingegnerizzazione dei prompt, con conseguente raffinamento delle risposte e regole guida per l’LLM è stato ispirato al tipico approccio **ReAct**, il quale è pensato per analisi di problemi complessi, attraverso ragionamento e azione. [16]

2.12 Provider LLM

Le prime analisi sono state condotte utilizzando un modello di dimensioni ridotte, eseguito tramite **Ollama** (*llama3:8b*) [15] attraverso *API REST* locali con richieste *HTTP POST*. L’esecuzione su un dispositivo con risorse limitate ha però comportato tempi di risposta elevati e risultati poco accurati. Si è quindi passati a esplorare soluzioni *on demand* per superare i limiti computazionali dell’esecuzione locale. Tra queste è stato scelto **Groq** [4], utilizzando il piano gratuito messo a disposizione ed il modello *meta-llama/llama-4-scout-17b-16e-instruct*. Il modello è stato interrogato tramite la specifica API *OpenAI*, utilizzando autenticazione basata su *API key* salvata come variabile d’ambiente. I risultati iniziali hanno mostrato discrete prestazioni, ma le limitazioni dovute al tetto massimo di *Token per Day* (TPD), insieme al *rate limit* tra una chiamata e l’altra hanno reso necessario individuare una soluzione alternativa. La **soluzione definitiva** adottata per condurre le analisi finali è stata **Azure** [12] sfruttando in particolare il modello *GPT-4o*, che offre anche capacità avanzate di *reasoning*. È stato utilizzato il servizio dedicato **Azure OpenAI**, effettuando le

chiamate tramite *SDK* (Software Development Kit) anziché tramite richieste *HTTP raw*, con autenticazione basata su *API key*. Avendo a disposizione un credito massimo limitato, previsto dal piano standard di Azure, si è reso necessario eseguire le analisi con LLM su un sottoinsieme dei commit rispetto all'intero dataset. Tale sottoinsieme è stato selezionato in modo **stratificato** rispetto alla distribuzione completa, garantendo comunque l'analisi di ogni esercizio per ciascuno studente che ha partecipato alla relativa esercitazione. Analogamente, le analisi sono state ripetute attraverso il modello *GPT-OSS-20b*, eseguito su un server remoto accessibile tramite la **VPN dell'Università** mediante *FortiClient*. Le richieste al modello sono state effettuate tramite *SDK OpenAI* con *base URL* personalizzato, utilizzando un endpoint compatibile con le API *OpenAI* ospitato all'interno della rete universitaria. Per rendere più stabile l'esecuzione delle numerose analisi, il sistema è stato configurato con meccanismi di *retry*, gestione dei *timeout* e controllo delle richieste concorrenti, così da compensare eventuali latenze dovute alla connessione VPN o al carico del server remoto. In questo modo sono stati raccolti risultati anche attraverso un ulteriore modello, avente caratteristiche differenti.

2.13 Flusso di analisi completo

L'intero flusso di analisi che viene seguito per ogni esercitazione, studente, esercizio e relativo commit sottomesso può essere illustrato nella

Figura 2.3 è descritto attraverso diverse fasi operative:

FASI 1-5 (Loop interno per ogni commit candidato, eseguito da script bash):

1. Commit checkout
2. Test Injection
3. Compilazione e test dinamici (script Bash)
4. Analisi statica (Semgrep)
5. Classificazione della categoria di fallimento

FASI 6-7 (Nel main Python):

6. Selezione proporzionale del commit (valuta i risultati 1-5)
7. Checkout del commit selezionato

FASI 8-9 (Analisi LLM):

8. Analisi LLM primario (diagnosi)
9. Valutazione LLM giudice (grading semantico)

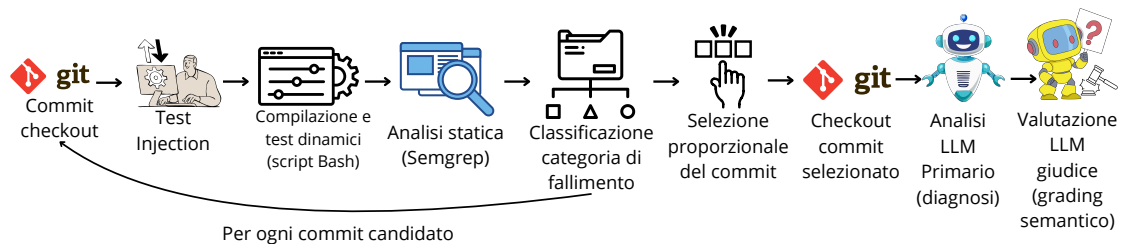


Figura 2.3: Flusso di analisi completo

2.14 Salvataggio risultati

I risultati vengono salvati in modo incrementale durante l'analisi in file **JSON** separati per esercitazione, effettuando un *parsing* delle risposte degli LLM per i corrispondenti campi, ottenendo record strutturati in questo modo:

```
{
  "student": "nome_studente",
  "exercise": "nome_esercizio",
  "commit_analyzed": "hash_commit",
  "failure_category": "compile_failure | timeout | crash | ipc_leak
    | dynamic_failure | static_failure | correct",
  "primary_model": "modello_usato",
  "judge_model": "modello_giudice",
  "compile_success": true/false,
  "test_success": true/false,
  "stdout": "output_stdout",
  "stderr": "output_stderr",
  "test_feedback": "feedback_test_normalizzato",
  "program_output": "output_programma",
  "static_warnings": [{file, line, message}, ...],
  "llm_Output_Correct": "YES/NO",
  "llm_Output_Diagnosis": "diagnosi_output",
  "llm_Code_Correct": "YES/NO",
  "llm_Code_Diagnosis": "diagnosi_codice",
  "judge_Output_Correct": "YES/NO",
  "judge_Output_Motivation": "motivazione_giudice",
  "judge_Code_Correct": "YES/NO",
  "judge_Code_Motivation": "motivazione_giudice"
}
```

Capitolo 3

Risultati

Le analisi sono state condotte per ciascuna delle **cinque esercitazioni** e articolate in due componenti distinte. In primo luogo, l'intero insieme dei commit è stato esaminato applicando esclusivamente test dinamici e statici, senza il ricorso a LLM, con l'obiettivo di ottenere una caratterizzazione completa dei dati e di ricostruire la distribuzione complessiva delle categorie di fallimento. Successivamente, è stata eseguita l'analisi completa basata su LLM su un sottoinsieme di commit selezionato tramite campionamento stratificato, mantenendo il vincolo di includere almeno un esercizio per ogni studente che avesse effettuato almeno un commit. Le analisi effettuate possono essere così riepilogate:

- Grafico aggregato relativo a tutte le esercitazioni e all'intero insieme dei commit.
- Grafico aggregato relativo a tutte le esercitazioni e ai soli commit analizzati mediante LLM.

- Calcolo delle statistiche per esercitazione ed esercizio sull'intero insieme dei commit.
- Grafico della distribuzione delle failure category suddivisa per esercitazione.
- Grafico della distribuzione delle failure category per esercitazione, considerando esclusivamente i commit sottoposti ad analisi con LLM.
- Grafici dell'analisi LLM per esercitazione, suddivisi nelle categorie `dynamic_failure`, `static_failure` e `correct`.
- Stima dell'accuratezza degli LLM articolata per esercitazione, singolo esercizio e failure category.
- Analisi statica condotta tramite strumenti dedicati e basata su un insieme di regole generiche.

Alcuni esempi delle analisi effettuate mediante LLM, suddivisi per esercitazione e categoria di fallimento, possono essere consultati sul repository di riferimento in un apposito documento [appendice.\[2\]](#)

3.1 Distribuzione globale

L'analisi globale è stata realizzata considerando i commit sottomessi da ciascuno studente, includendo esclusivamente quelli che introduco-

no modifiche effettive. L'identificazione dei commit rilevanti è avvenuta tramite l'applicazione di un filtro di percorso nel comando "`git rev-list main - .`", che limita l'elaborazione ai soli commit che hanno modificato file all'interno della directory dell'esercizio. In questo modo sono stati esclusi automaticamente i commit non pertinenti, evitando la raccolta di implementazioni non significative.

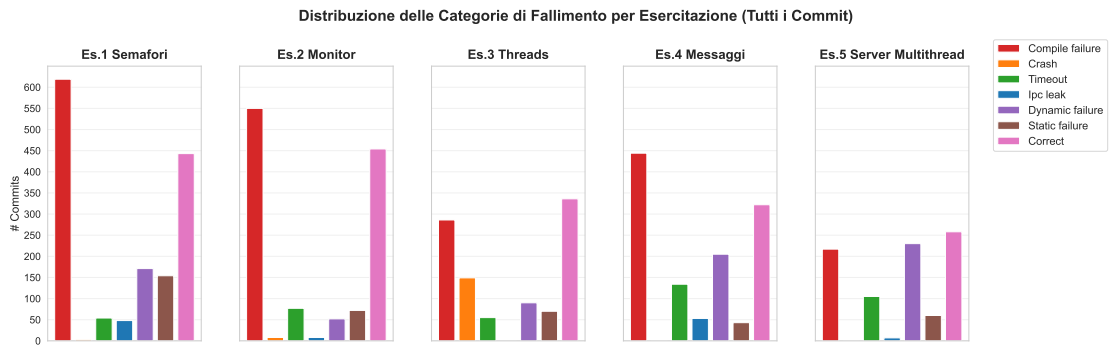


Figura 3.1: Distribuzione globale delle categorie di fallimento

3.2 Raccolta dei commit analizzati con LLM

L'analisi mantiene il vincolo di raccogliere tutti gli esercizi per ciascuno studente, pur consentendo la selezione di commit differenti. La scelta dei commit da includere avviene tramite un **algoritmo greedy locale** che, ad ogni iterazione, osserva la **distribuzione globale target** e, per ogni esercizio di ogni studente, valuta tutti i commit disponibili selezionando quello che produce una distribuzione delle categorie più vicina al target stesso. La vicinanza tra la distribuzione del commit candidato e quella globale viene misurata tramite la **distanza**

L1 (o *distanza di Manhattan*), calcolata sui conteggi delle categorie analizzate. La formula utilizzata è:

$$L1(\mathbf{x}, \mathbf{t}) = \sum_{i=1}^k |x_i - t_i|$$

dove:

- $\mathbf{x} = (x_1, x_2, \dots, x_k)$ è la **distribuzione del commit candidato**;
- $\mathbf{t} = (t_1, t_2, \dots, t_k)$ è la **distribuzione target globale**;
- k è il numero di categorie considerate;
- $|x_i - t_i|$ rappresenta la **differenza assoluta** tra commit e target per la categoria i .

L'algoritmo seleziona quindi, ad ogni passo, il commit che **minimizza** tale distanza, garantendo una composizione del campione quanto più possibile proporzionale alla distribuzione complessiva.

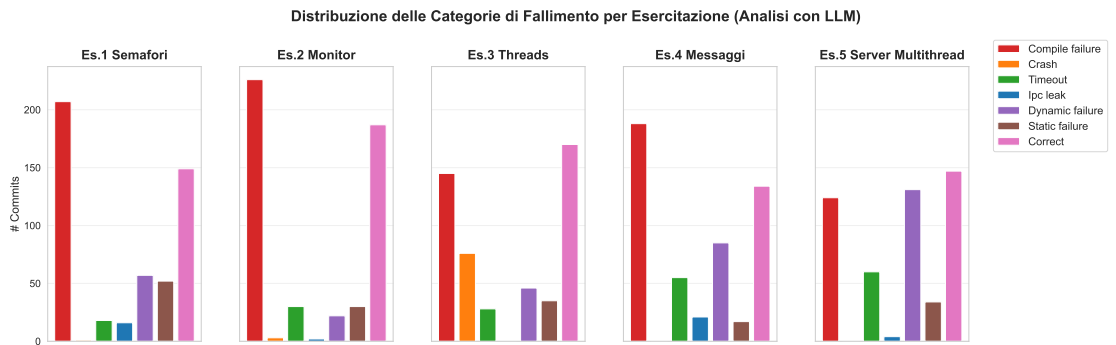


Figura 3.2: Distribuzione delle categorie di fallimento relativa a sottinsieme sottoposto ad analisi con LLM

3.3 Risultati dell'analisi per ciascuna esercitazione, modello GPT-4o

Lo studio è stato dunque realizzato per ciascuna esercitazione, attraverso il calcolo di statistiche dedicate e la valutazione dei risultati ottenuti mediante l'utilizzo degli LLM e dei relativi prompt. Il comportamento dei modelli è stato esaminato tramite una serie di grafici che ne illustrano le performance nelle diverse categorie considerate. L'accuratezza degli LLM è stata stimata formulando quattro quesiti di valutazione, adattando le etichette corretto e scorretto in funzione della specifica failure category:

1. LLM valuta correttamente se l'output è corretto?
2. LLM valuta correttamente se il codice è corretto?
3. Quando LLM rileva output errato, la diagnosi è corretta?
4. Quando LLM rileva codice errato, la diagnosi è corretta?

3.3.1 Risultati esercitazione 1 - Semafori

Le principali statistiche relative all'esercitazione sono le seguenti:

- Numero di studenti con almeno 1 commit: 132
- Numero totale di commit git: 1431
- Numero totale di modifiche agli esercizi: 1492

- Numero di modifiche analizzate tramite LLM: 500

In relazione alla distribuzione globale, sono state raccolte le seguenti statistiche per ciascun esercizio dell'esercitazione:

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct
calcolo_parallelo_su_un_vettore_condiviso	158	3	1	6	45	72	136
coppia_di_buffer	161	0	18	8	16	39	98
lettori-scrittori_con_semafori_su_una_coppia_di_valori_condivisa	145	0	7	23	47	43	112
simulazione_di_un_disco_con_un_vettore_circolare	155	0	28	11	63	0	97

Tabella 3.1: Conteggio delle categorie di fallimento per ciascun esercizio

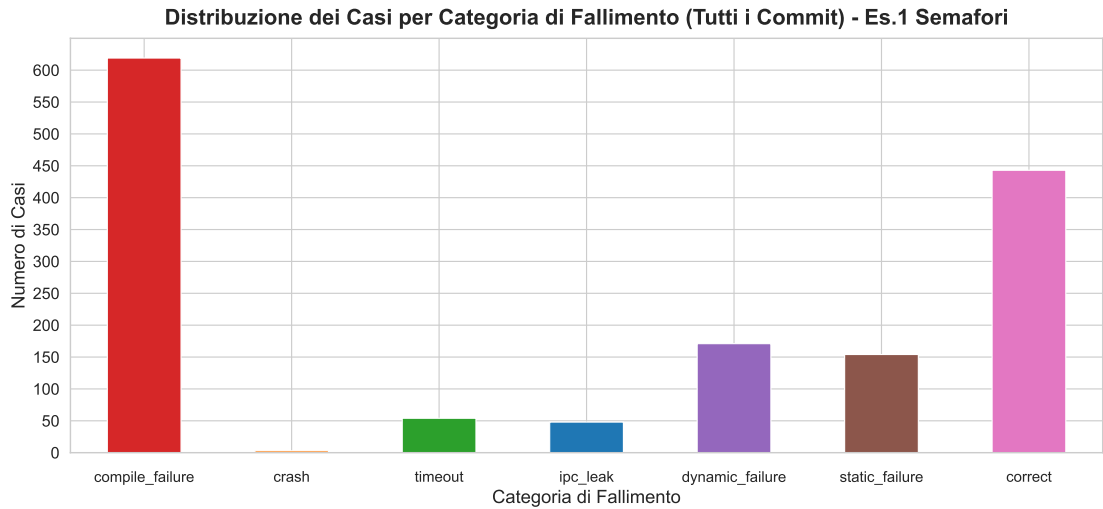


Figura 3.3: Distribuzione globale delle categorie di fallimento per esercitazione 1

Risultati Analisi mediante LLM

La distribuzione dei commit analizzati tramite LLM, selezionati in proporzione alla distribuzione completa, è riportata di seguito:

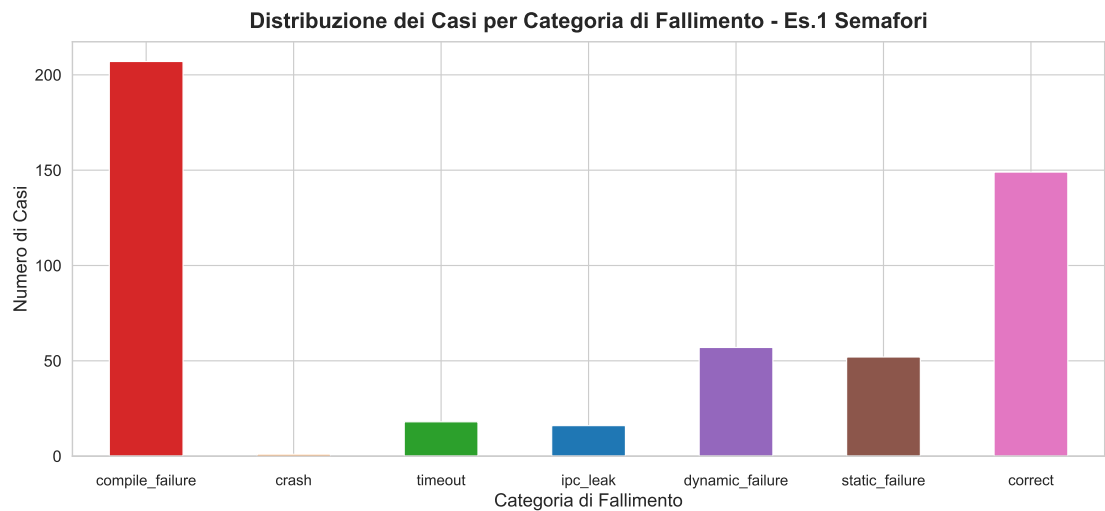


Figura 3.4: Distribuzione dei fallimenti nei commit analizzati con LLM – esercitazione 1

I risultati prodotti dagli LLM sono stati analizzati con riferimento alle categorie **dynamic_failure**, **static_failure** e **correct**, e sono rappresentati nei grafici riportati di seguito.

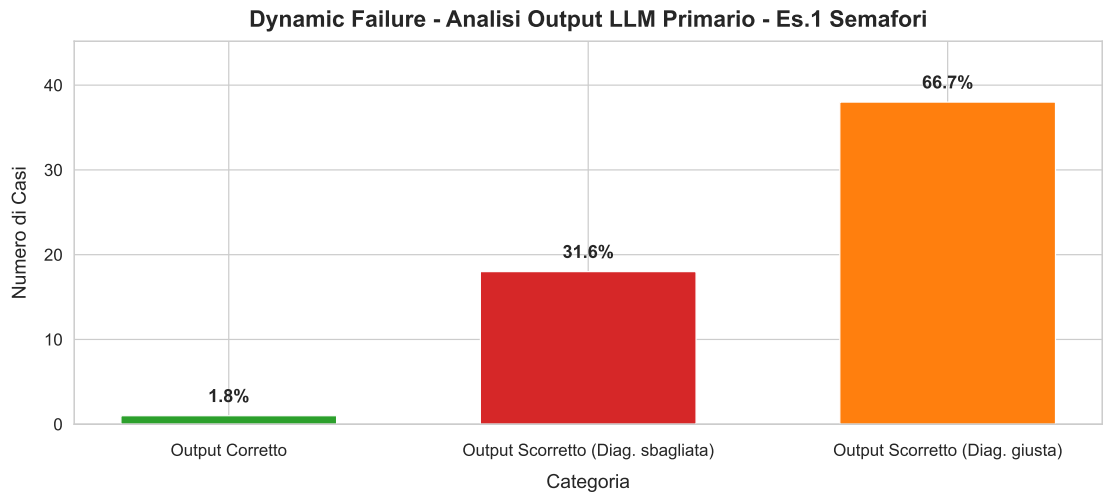


Figura 3.5: Risultati analisi con LLM per output, categoria: Dynamic failure - esercitazione 1.

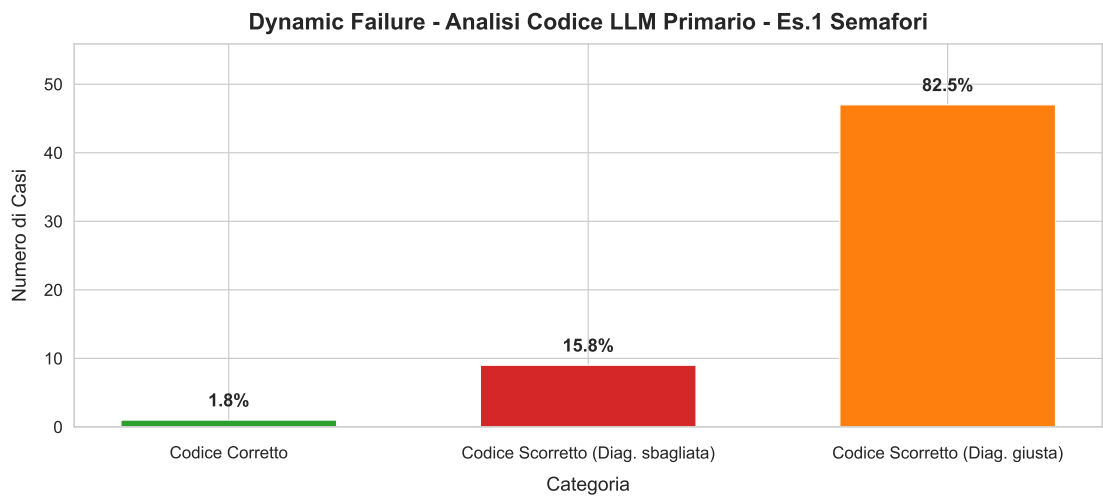


Figura 3.6: Risultati analisi con LLM per codice, categoria: Dynamic failure - esercitazione 1.

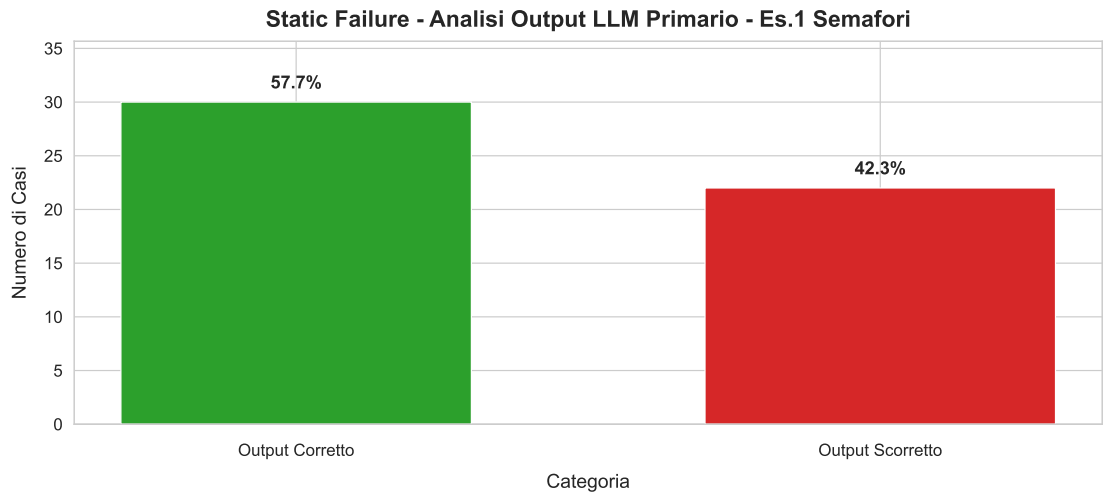


Figura 3.7: Risultati analisi con LLM per output, categoria: Static failure - esercitazione 1.

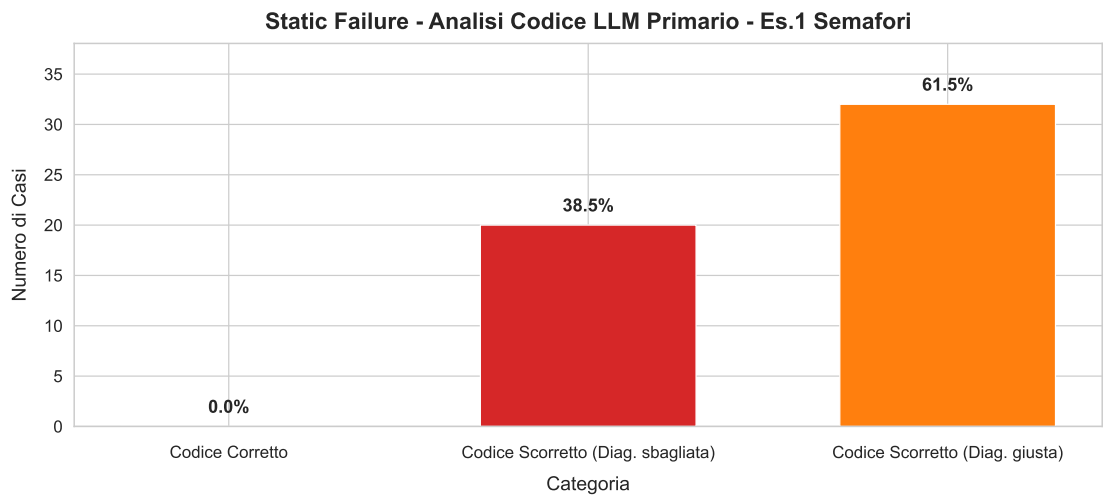


Figura 3.8: Risultati analisi con LLM per codice, categoria: Static failure - esercitazione 1.

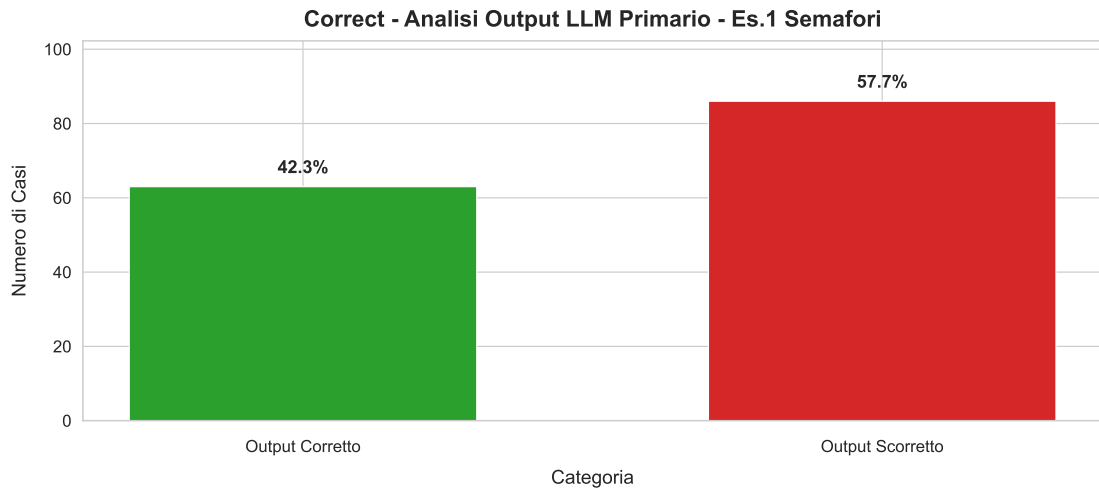


Figura 3.9: Risultati analisi con LLM per output, categoria: Correct - esercitazione 1.

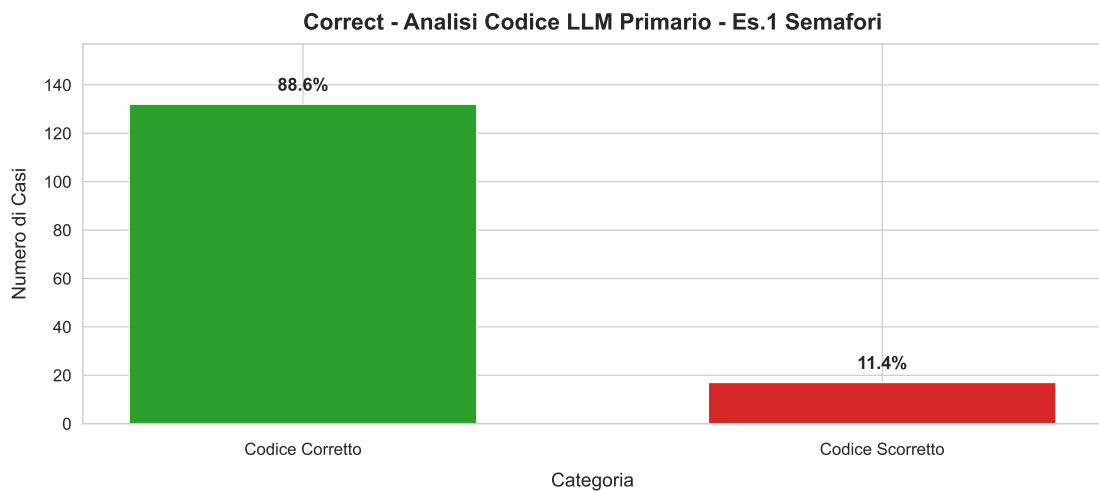


Figura 3.10: Risultati analisi con LLM per codice, categoria: Correct - esercitazione 1.

Descrizione risultati

Le categorie **crash**, **timeout** e **ipc_leak** mostrano un'accuratezza perfetta, segno che l'LLM primario riconosca tali fallimenti in modo ottimale. I casi di **dynamic_failure** vengono valutati correttamente, ma le diagnosi riguardanti l'output risultano meno accurate, con il

67,9%. Gli **static_failure** mostrano il divario più evidente: la valutazione dell'output è poco accurata (57,7%), mentre quella del codice è perfetta, anche se con diagnosi non sempre precise, evidenziando maggiore efficacia nell'analisi statica piuttosto che nell'interpretazione dell'output generato dall'esecuzione. Infine i programmi classificati come **correct** presentano valutazione dell'output piuttosto imprecisa (42,3%), e valutazione del codice con percentuale molto più elevata (88,6%), ma relative diagnosi altamente accurate.

Failure category	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
crash	N/A	100.0% (1/1)	N/A	100.0% (1/1)
timeout	N/A	100.0% (18/18)	N/A	100.0% (18/18)
ipc_leak	N/A	100.0% (16/16)	N/A	100.0% (16/16)
dynamic_failure	98.2% (56/57)	98.2% (56/57)	67.9% (38/56)	83.9% (47/56)
static_failure	57.7% (30/52)	100.0% (52/52)	100.0% (30/30)	61.5% (32/52)
correct	42.3% (63/149)	88.6% (132/149)	100.0% (63/63)	100.0% (132/132)

Tabella 3.2: Accuratezza per categoria di fallimento

Per quanto riguarda gli esercizi specifici, emergono notevoli differenze: **calcolo parallelo**, **lettori-scrittori** e **simulazione del disco** mostrano diagnosi output poco affidabili, mentre per **coppia di buffer** risultano decisamente più accurate. Allo stesso modo però tale esercizio evidenzia diagnosi codice peggiori, nonostante l'elevata accuratezza nelle valutazioni.

Exercise	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
calcolo_parallelo_su_un_vettore_condiviso	76.4% (94/123)	97.6% (123/126)	66.7% (6/9)	90.0% (36/40)
coppia_di_buffer	80.2% (93/116)	96.0% (120/125)	90.0% (9/10)	55.9% (19/34)
lettori-scrittori_con_semafori_su_una_coppia_di_valori_condivisa	71.9% (82/114)	92.8% (116/125)	60.0% (9/15)	74.3% (26/35)
simulazione_di_un_disco_con_un_vettore_circolare	76.8% (86/112)	99.2% (123/124)	63.6% (14/22)	97.1% (33/34)

Tabella 3.3: Accuratezza per esercizio specifico

3.3.2 Risultati esercitazione 2 - Monitor

Le principali statistiche relative all'esercitazione sono le seguenti:

- Numero di studenti con almeno 1 commit: 128
- Numero totale di commit git: 1178
- Numero totale di modifiche agli esercizi: 1221
- Numero di modifiche analizzate tramite LLM: 500

In relazione alla distribuzione globale, sono state raccolte le seguenti statistiche per ciascun esercizio dell'esercitazione:

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct
lettori-scrittori_con_monitor_e processi	147	0	20	5	3	3	153
prelievi_multipli	132	3	17	3	11	44	89
produttore-consumatore_con_priorita	135	4	21	0	18	25	110
simulazione_di_un_disco_con_un_vettore_circolare	136	1	19	0	20	0	102

Tabella 3.4: Conteggio delle categorie di fallimento per ciascun esercizio

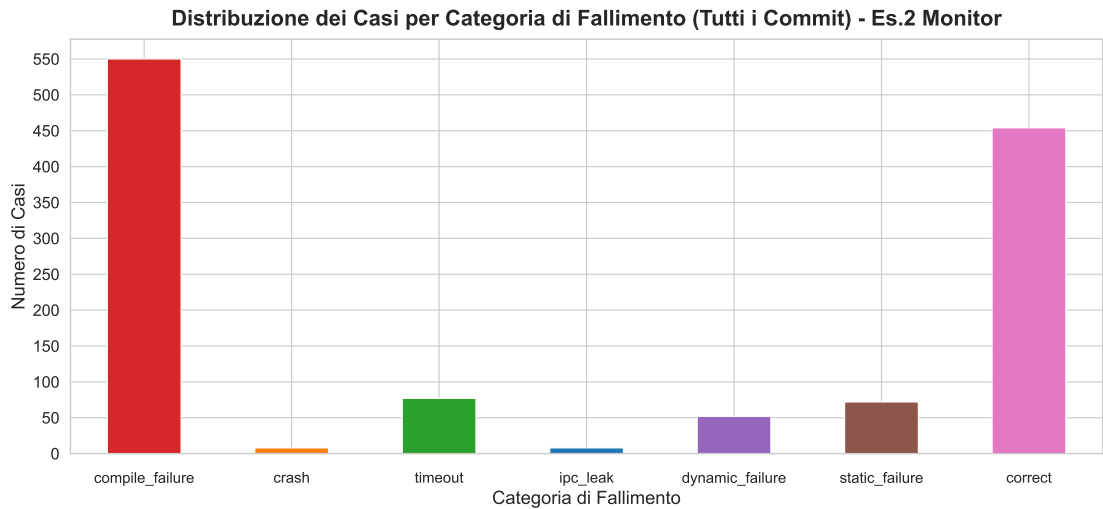


Figura 3.11: Distribuzione globale delle categorie di fallimento per esercitazione 2

Risultati Analisi mediante LLM

La distribuzione dei commit analizzati tramite LLM, selezionati in proporzione alla distribuzione completa, è riportata di seguito:

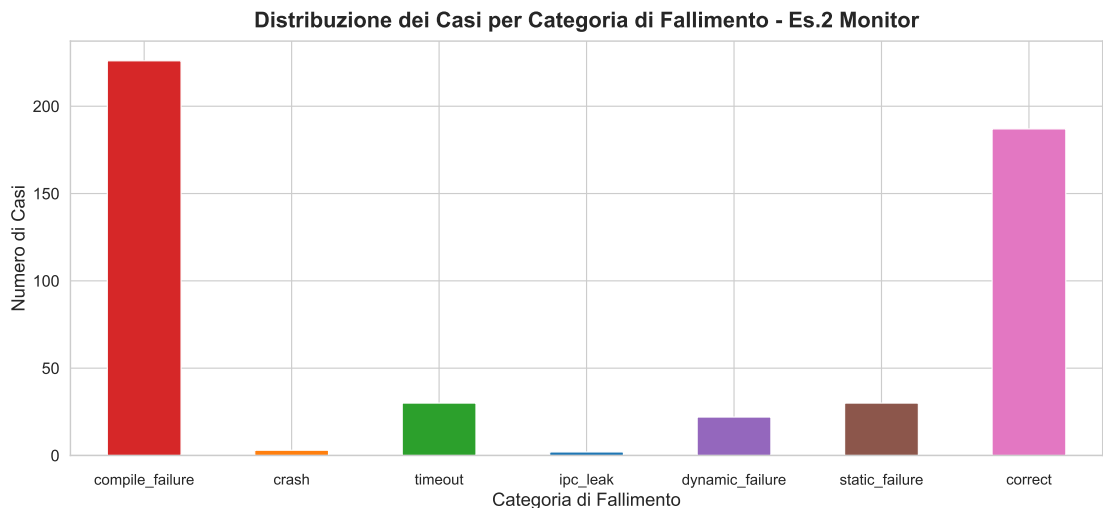


Figura 3.12: Distribuzione dei fallimenti nei commit analizzati con LLM – esercitazione 2

I risultati prodotti dagli LLM sono stati analizzati con riferimento

alle categorie `dynamic_failure`, `static_failure` e `correct`, e sono rappresentati nei grafici riportati di seguito.

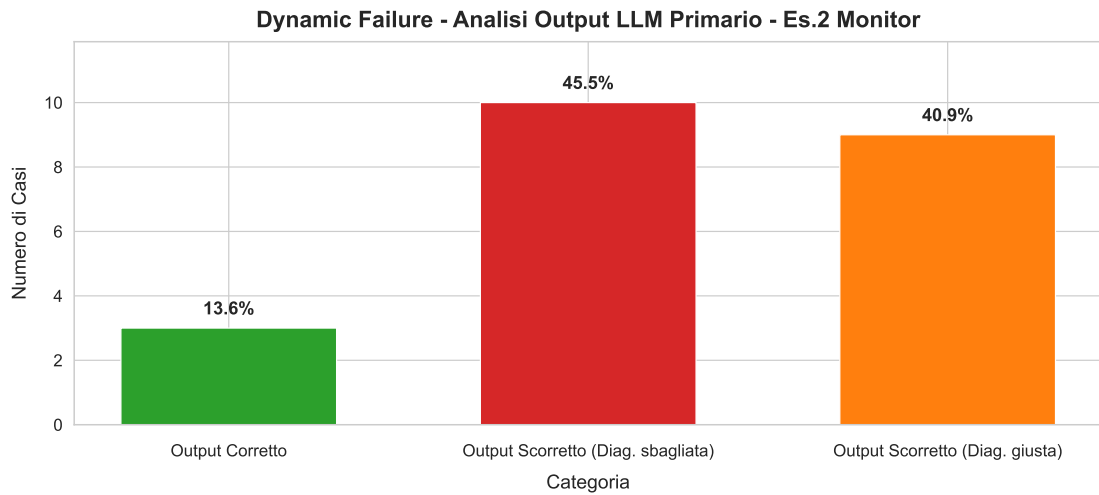


Figura 3.13: Risultati analisi con LLM per output, categoria: Dynamic failure - esercitazione 2.

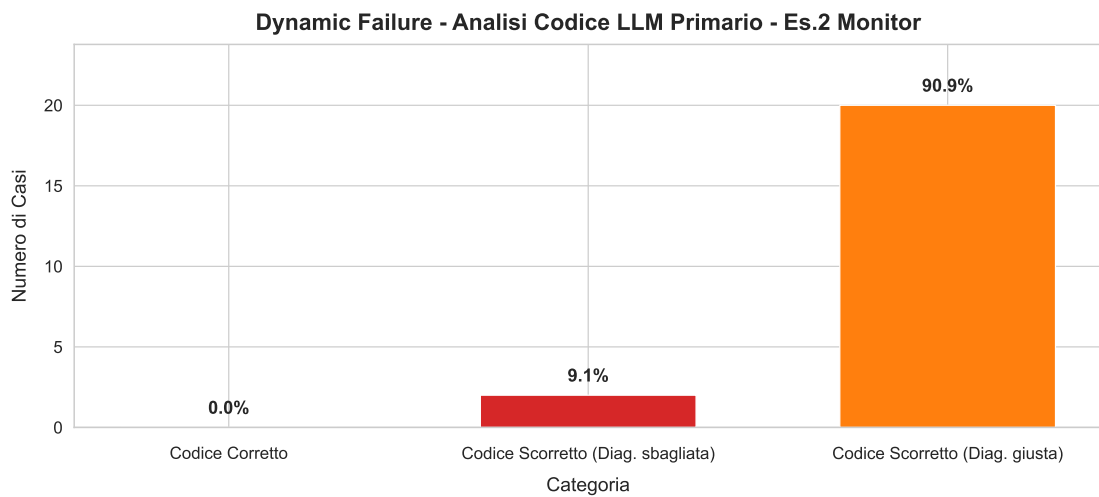


Figura 3.14: Risultati analisi con LLM per codice, categoria: Dynamic failure - esercitazione 2.

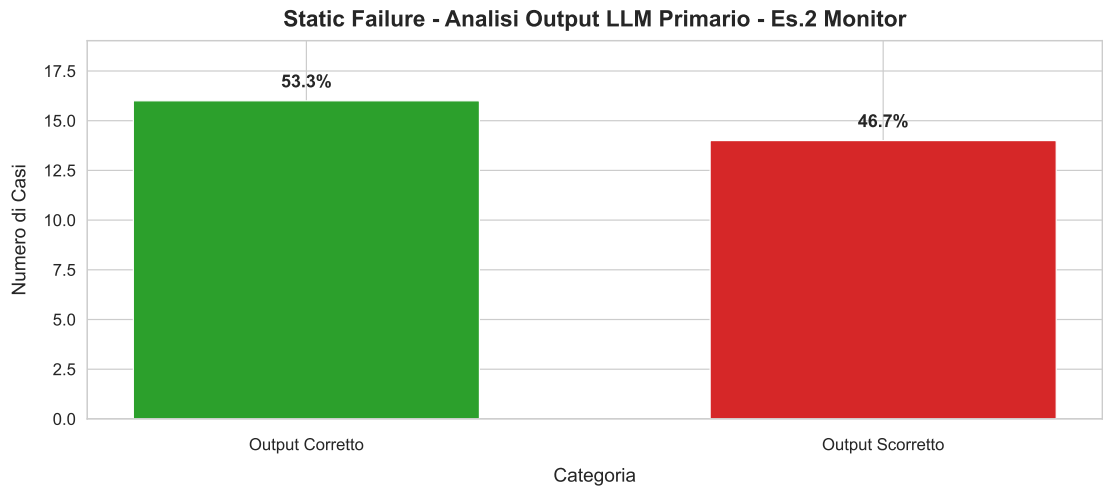


Figura 3.15: Risultati analisi con LLM per output, categoria: Static failure - esercitazione 2.

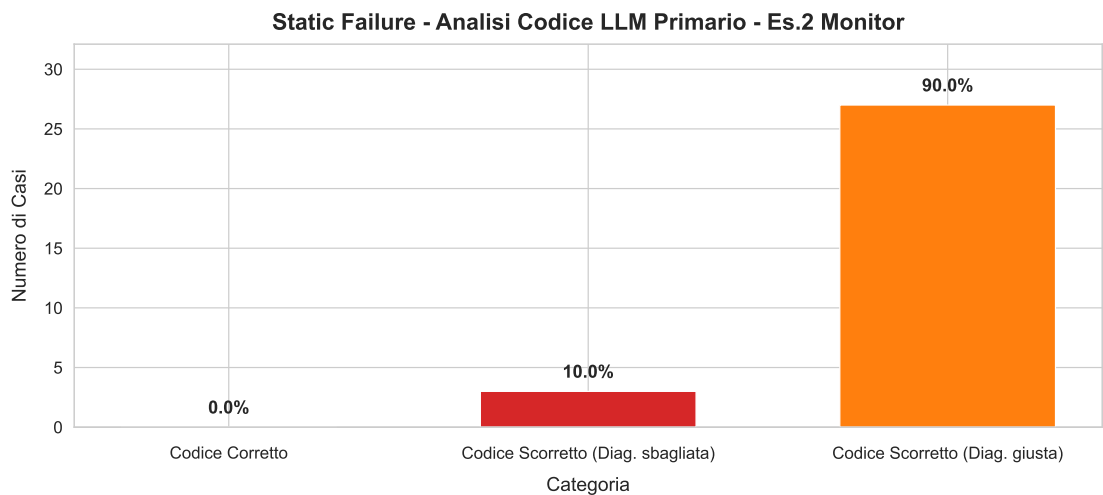


Figura 3.16: Risultati analisi con LLM per codice, categoria: Static failure - esercitazione 2.

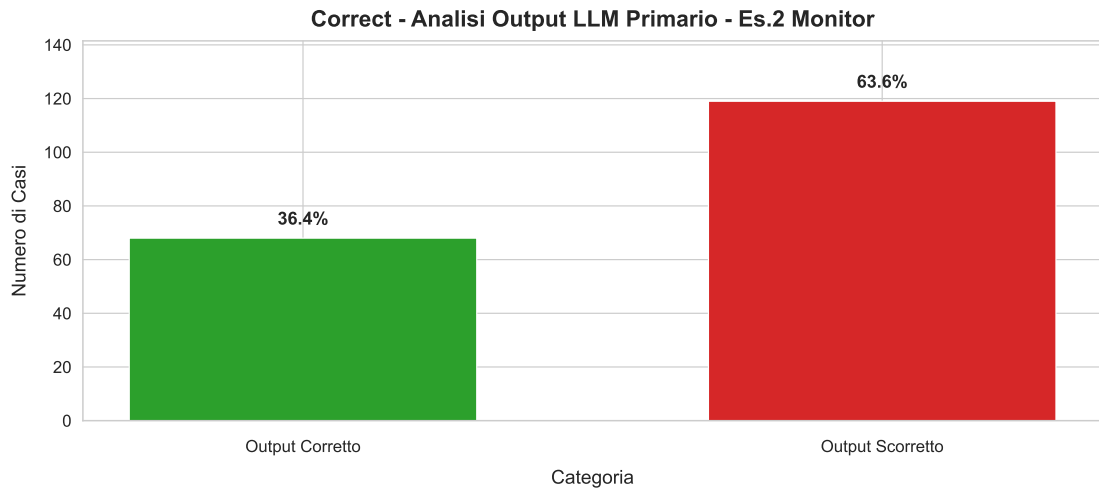


Figura 3.17: Risultati analisi con LLM per output, categoria: Correct - esercitazione 2.

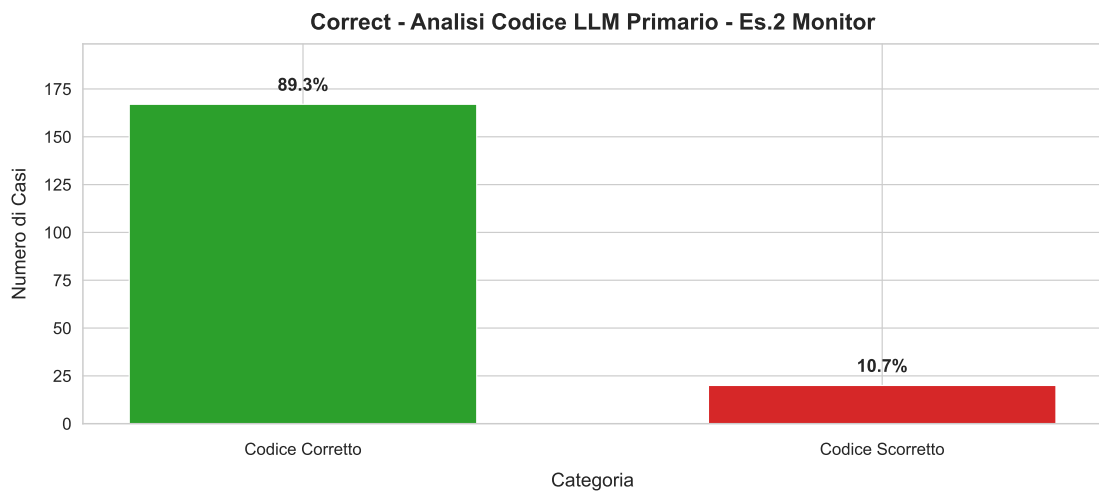


Figura 3.18: Risultati analisi con LLM per codice, categoria: Correct - esercitazione 2.

Descrizione risultati

Le categorie **crash**, **timeout** e **ipc_leak** confermano un'accuratezza perfetta sia nella valutazione sia nella diagnosi del codice, come accaduto per l'esercitazione 1, e visibile in Tabella 3.2. I casi di **dynamic_failure** presentano invece un forte divario: la valutazione è

molto accurata (soprattutto sul codice), ma la diagnosi sull'output risulta più bassa (47,4%), segno che il modello ha maggiore difficoltà ad individuare precisamente la causa del fallimento dinamico. Gli **static_failure** mostrano una modesta valutazione dell'output (53,3%), a discapito di valutazione del codice perfetta e relativa diagnosi piuttosto affidabile. Infine i programmi classificati come **correct** presentano valutazione dell'output piuttosto imprecisa (36,4%), e valutazione del codice con percentuale molto più elevata (89,3%), ma relative diagnosi altamente accurate.

Failure category	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
crash	N/A	100.0% (3/3)	N/A	100.0% (3/3)
timeout	N/A	100.0% (30/30)	N/A	100.0% (30/30)
ipc_leak	N/A	100.0% (2/2)	N/A	100.0% (2/2)
dynamic_failure	86.4% (19/22)	100.0% (22/22)	47.4% (9/19)	90.9% (20/22)
static_failure	53.3% (16/30)	100.0% (30/30)	100.0% (16/16)	90.0% (27/30)
correct	36.4% (68/187)	89.3% (167/187)	100.0% (68/68)	100.0% (167/167)

Tabella 3.5: Accuratezza per categoria di fallimento

Per quanto riguarda gli esercizi specifici, emergono notevoli differenze. **Lettori-scrittori con monitor** mostra una valutazione dell'output pari al 49,6%, mentre la valutazione del codice rimane buona: il modello non riesce ad interpretare in modo affidabile il comportamento concorrente per quest'esercizio, però è in grado di valutare la struttura del codice. **Prelievi multipli** invece si è dimostrato più semplice da valutare, con accuratezza del 85,8% sull'output, ma con diagnosi comunque debole. **Produttore-consumatore con priorità** mostra ancora una volta valutazione dell'output piuttosto discreta e valuta-

zione del codice molto alta. **Simulazione del disco** mostra infine risultati opposti, attraverso valutazione dell'output ottima ma diagnosi sull'output completamente imprecisa.

Exercise	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
lettori-scrittori_con_monitor_e_processi	49.6% (58/117)	88.0% (110/125)	66.7% (2/3)	100.0% (13/13)
prelievi_multipli	85.8% (103/120)	100.0% (127/127)	42.9% (3/7)	93.8% (30/32)
produttore-consumatore_con_priorita	58.4% (66/113)	96.0% (119/124)	57.1% (4/7)	93.5% (29/31)
simulazione_di_un_disco_con_un_vettore_circolare	87.0% (100/115)	100.0% (124/124)	0.0% (0/2)	90.9% (10/11)

Tabella 3.6: Accuratezza per esercizio specifico

3.3.3 Risultati esercitazione 3 - Threads

Le principali statistiche relative all'esercitazione sono le seguenti:

- Numero di studenti con almeno 1 commit: 124
- Numero totale di commit git: 1056
- Numero totale di modifiche agli esercizi: 986
- Numero di modifiche analizzate tramite LLM: 500

In relazione alla distribuzione globale, sono state raccolte le seguenti statistiche per ciascun esercizio dell'esercitazione:

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct
lettori-scrittori_su_piu_oggetti_monitor	5	136	23	0	3	37	153
produttore-consumatore_con_priorita	145	4	15	0	58	21	92
una_struttura_dati_stack_thread-safe	136	9	17	0	29	12	91

Tabella 3.7: Conteggio delle categorie di fallimento per ciascun esercizio

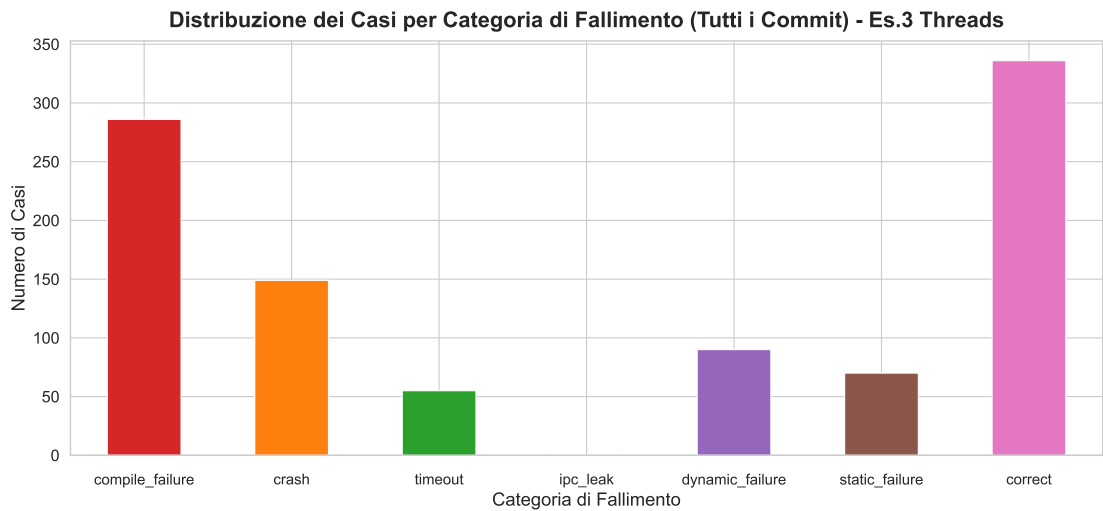


Figura 3.19: Distribuzione globale delle categorie di fallimento per esercitazione 3

Risultati Analisi mediante LLM

La distribuzione dei commit analizzati tramite LLM, selezionati in proporzione alla distribuzione completa, è riportata di seguito:

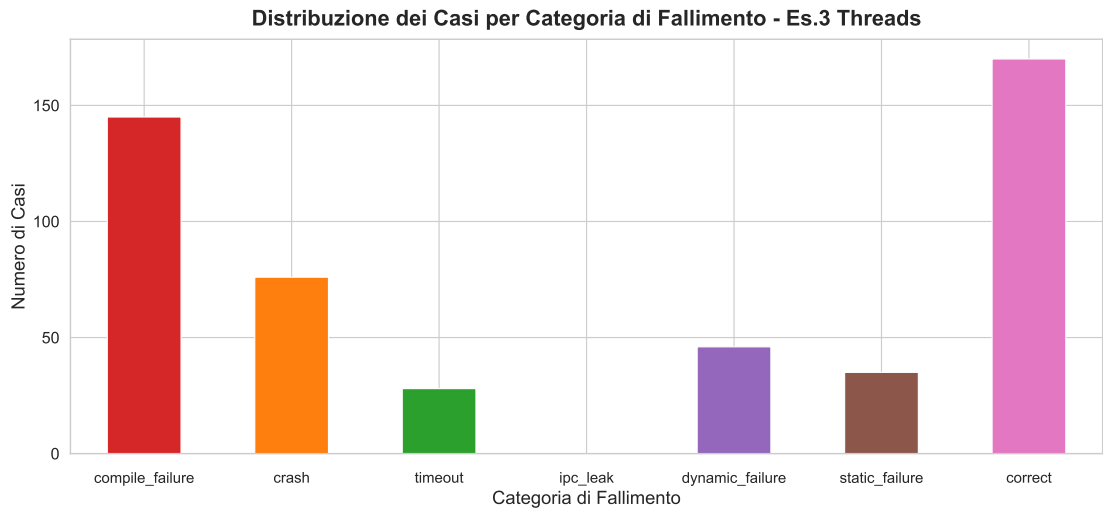


Figura 3.20: Distribuzione dei fallimenti nei commit analizzati con LLM – esercitazione 3

I risultati prodotti dagli LLM sono stati analizzati con riferimento alle categorie **dynamic_failure**, **static_failure** e **correct**, e sono rappresentati nei grafici riportati di seguito.

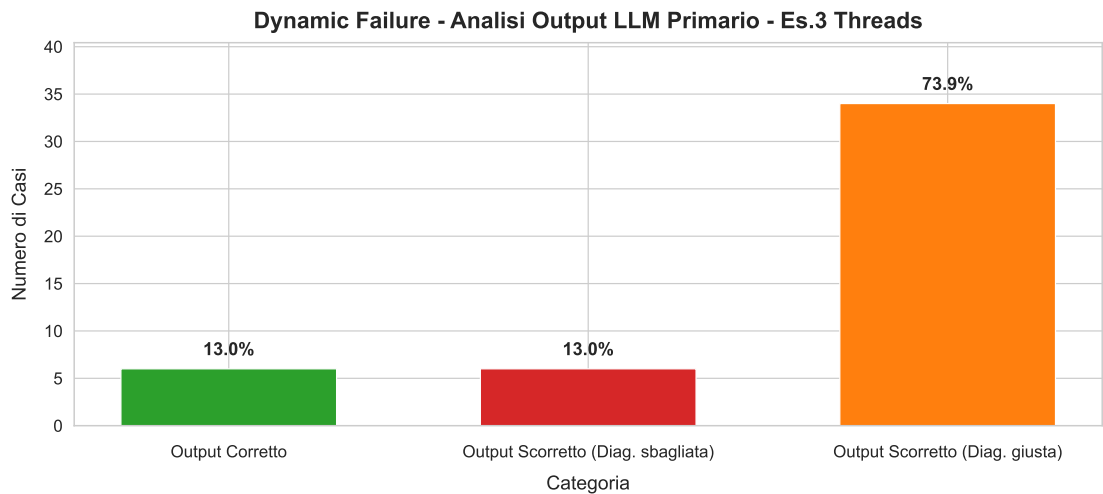


Figura 3.21: Risultati analisi con LLM per output, categoria: Dynamic failure - esercitazione 3.

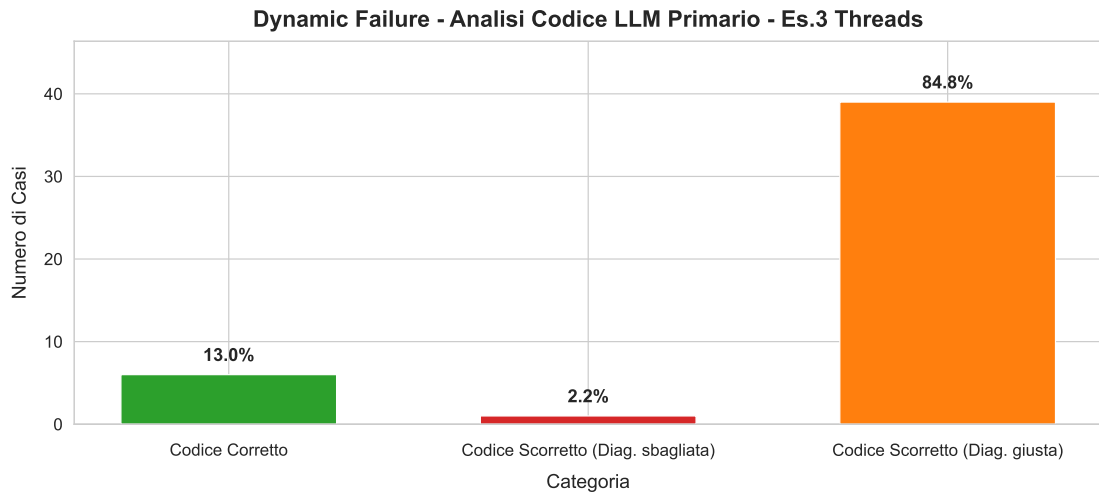


Figura 3.22: Risultati analisi con LLM per codice, categoria: Dynamic failure - esercitazione 3.

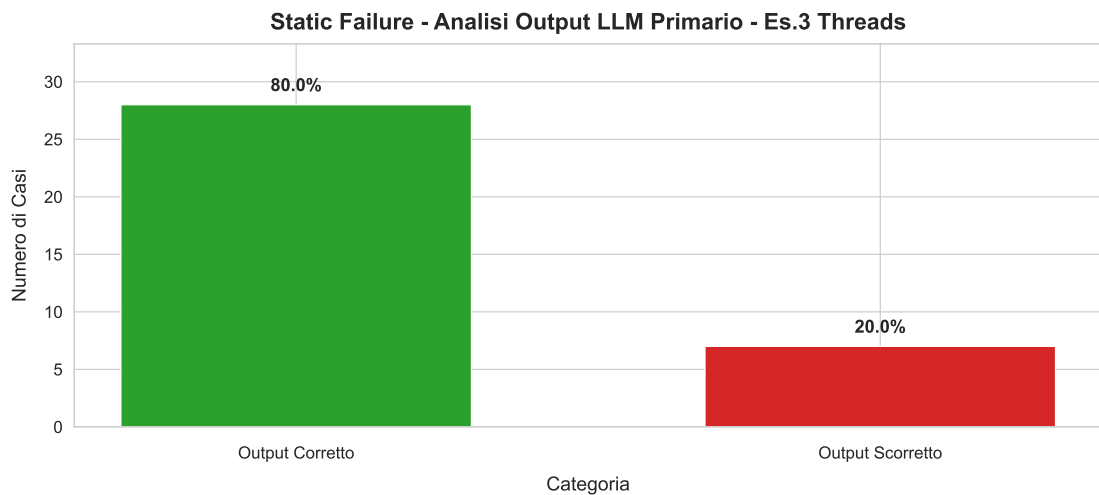


Figura 3.23: Risultati analisi con LLM per output, categoria: Static failure - esercitazione 3.

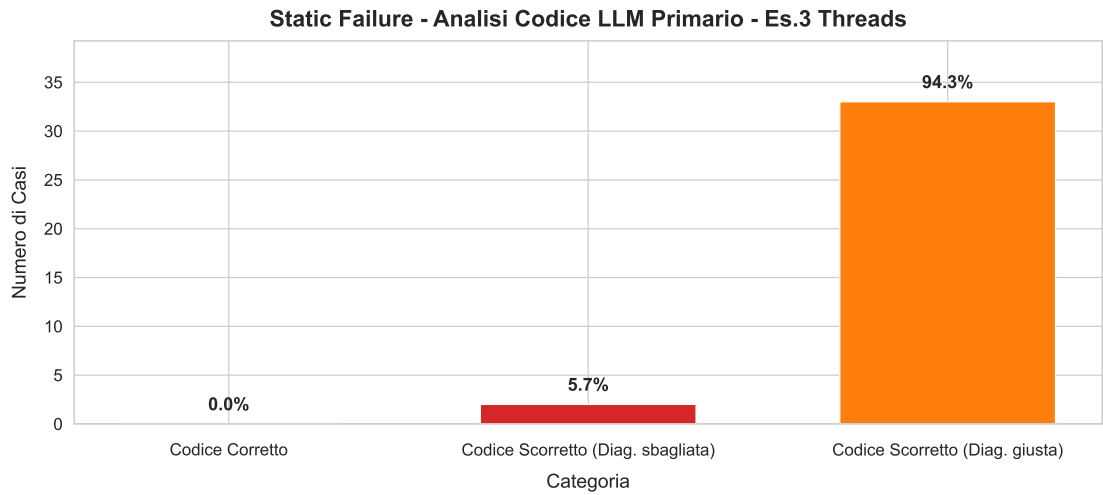


Figura 3.24: Risultati analisi con LLM per codice, categoria: Static failure - esercitazione 3.

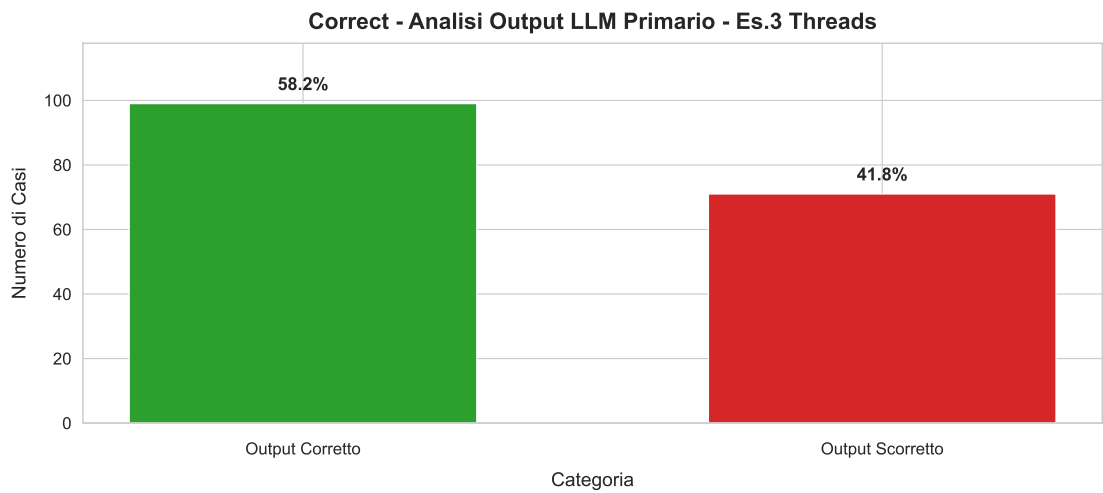


Figura 3.25: Risultati analisi con LLM per output, categoria: Correct - esercitazione 3.

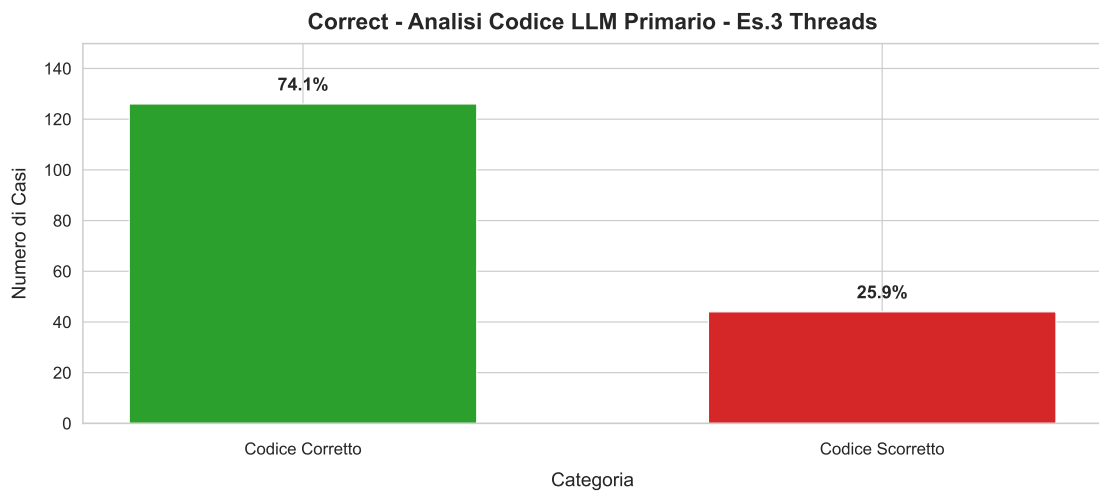


Figura 3.26: Risultati analisi con LLM per codice, categoria: Correct - esercitazione 3.

Descrizione risultati

Le categorie **crash** e **timeout** mostrano un'accuratezza perfetta sia nella valutazione che nella diagnosi, come accaduto per le precedenti esercitazioni. I casi di **dynamic_failure** presentano valutazioni ottime sia per l'output che per il codice, e diagnosi molto accurate. L'analisi dell'output rimane comunque leggermente meno affidabile, con accuratezza delle diagnosi del 85%, mostrando nuovamente maggiore difficoltà rispetto all'analisi del codice. Quest'ultimo aspetto è notevolmente visibile per gli **static_failure**, i quali mostrano valutazione dell'output meno precisa (80%), mentre quella del codice è perfetta. Infine i programmi classificati come **correct** presentano valutazione dell'output non particolarmente affidabile (58,2%), e valutazione del codice con percentuale più elevata (74,1%), ma relative diagnosi altamente accurate.

Failure category	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
crash	N/A	100.0% (76/76)	N/A	100.0% (76/76)
timeout	N/A	100.0% (28/28)	N/A	100.0% (28/28)
ipc_leak	N/A	N/A	N/A	N/A
dynamic_failure	87.0% (40/46)	87.0% (40/46)	85.0% (34/40)	97.5% (39/40)
static_failure	80.0% (28/35)	100.0% (35/35)	100.0% (28/28)	94.3% (33/35)
correct	58.2% (99/170)	74.1% (126/170)	100.0% (99/99)	100.0% (126/126)

Tabella 3.8: Accuratezza per categoria di fallimento

Per quanto riguarda gli esercizi specifici, emergono in questo caso comportamenti più equilibrati. Infatti, in generale, la valutazione output presenta un'accuratezza minima del 68,9% per **Lettori-scrittori su più oggetti monitor** e massima del 75,3% per **Una struttura dati stack thread-safe**, mentre invece l'analisi del codice mostra performance migliori, raggiungendo percentuali tra il 95 e 100% di accuratezza nelle diagnosi, dimostrando maggior abilità nell'analisi statica piuttosto che quella basata sull'output prodotto.

Exercise	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
lettori-scrittori_su_piu_oggetti_monitor	68.9% (51/74)	85.5% (142/166)	100.0% (3/3)	100.0% (117/117)
produttore-consumatore_con_priorita	73.8% (124/168)	92.5% (161/174)	77.8% (21/27)	95.0% (38/40)
una_struttura_dati_stack_thread-safe	75.3% (116/154)	91.9% (147/160)	100.0% (10/10)	95.5% (21/22)

Tabella 3.9: Accuratezza per esercizio specifico

3.3.4 Risultati esercitazione 4 - Messaggi

Le principali statistiche relative all'esercitazione sono le seguenti:

- Numero di studenti con almeno 1 commit: 117

- Numero totale di commit git: 1243
- Numero totale di modifiche agli esercizi: 1201
- Numero di modifiche analizzate tramite LLM: 500

In relazione alla distribuzione globale, sono state raccolte le seguenti statistiche per ciascun esercizio dell'esercitazione:

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct
grafo_delle_dipendenze	118	0	42	12	32	1	62
load_balancing	107	0	6	8	49	7	69
registro_distribuito	111	0	36	3	32	14	89
server_sincroni_multipli	108	0	50	30	92	21	102

Tabella 3.10: Conteggio delle categorie di fallimento per ciascun esercizio

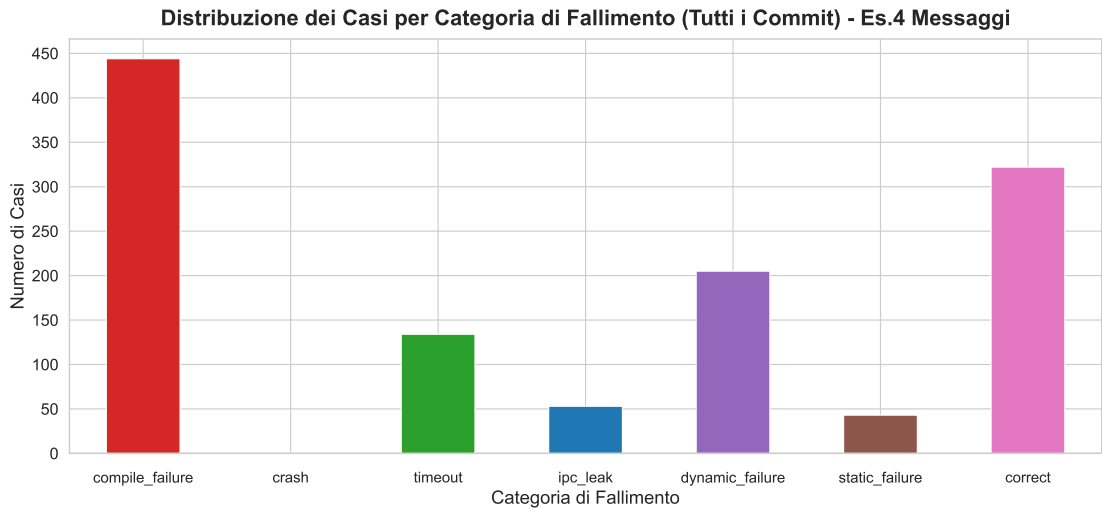


Figura 3.27: Distribuzione globale delle categorie di fallimento per esercitazione 4

Risultati Analisi mediante LLM

La distribuzione dei commit analizzati tramite LLM, selezionati in proporzione alla distribuzione completa, è riportata di seguito:

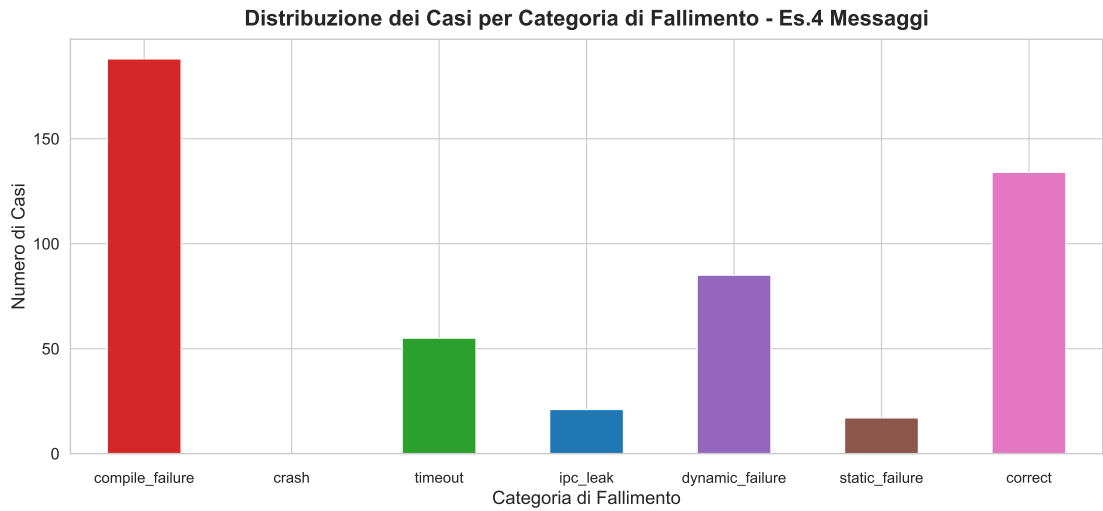


Figura 3.28: Distribuzione dei fallimenti nei commit analizzati con LLM – esercitazione 4

I risultati prodotti dagli LLM sono stati analizzati con riferimento alle categorie **dynamic_failure**, **static_failure** e **correct**, e sono rappresentati nei grafici riportati di seguito.

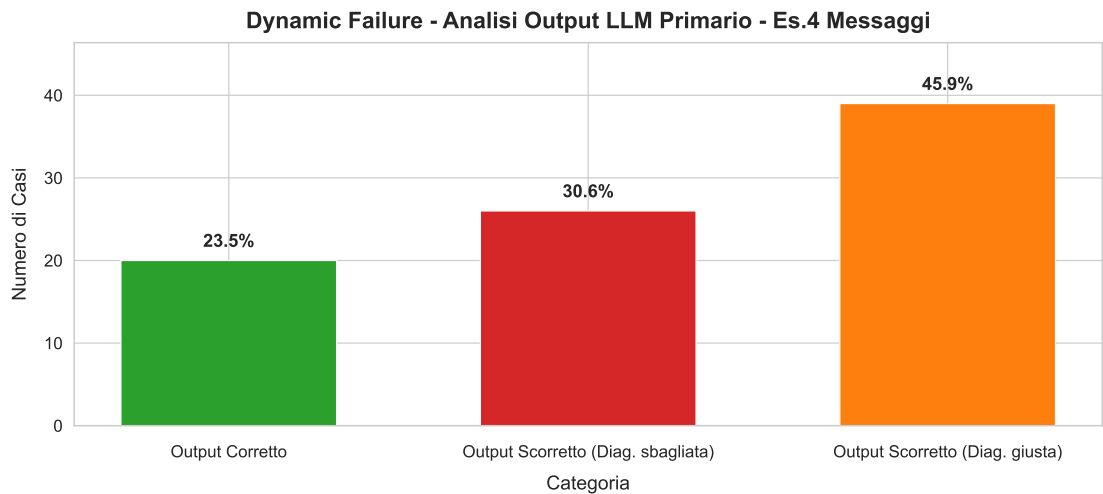


Figura 3.29: Risultati analisi con LLM per output, categoria: Dynamic failure - esercitazione 4.

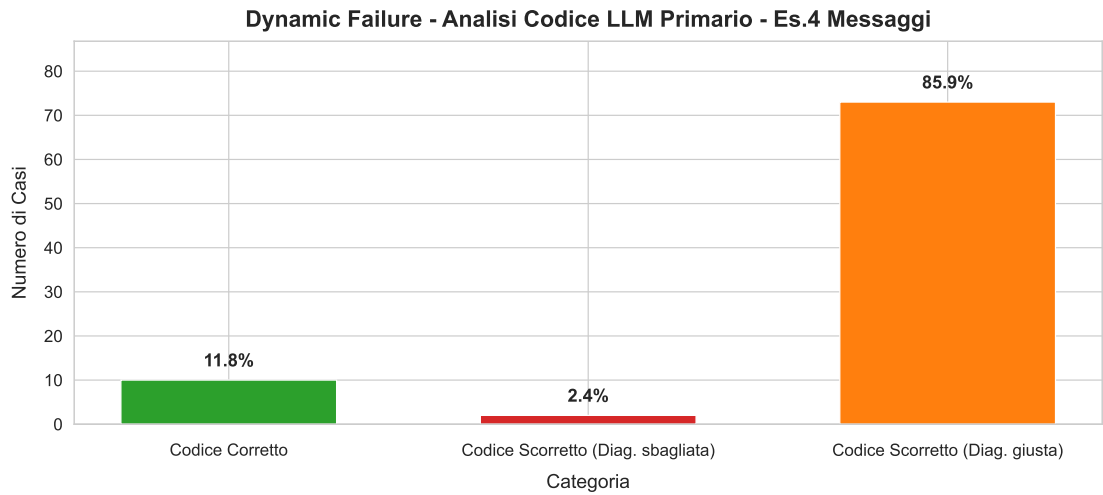


Figura 3.30: Risultati analisi con LLM per codice, categoria: Dynamic failure - esercitazione 4.

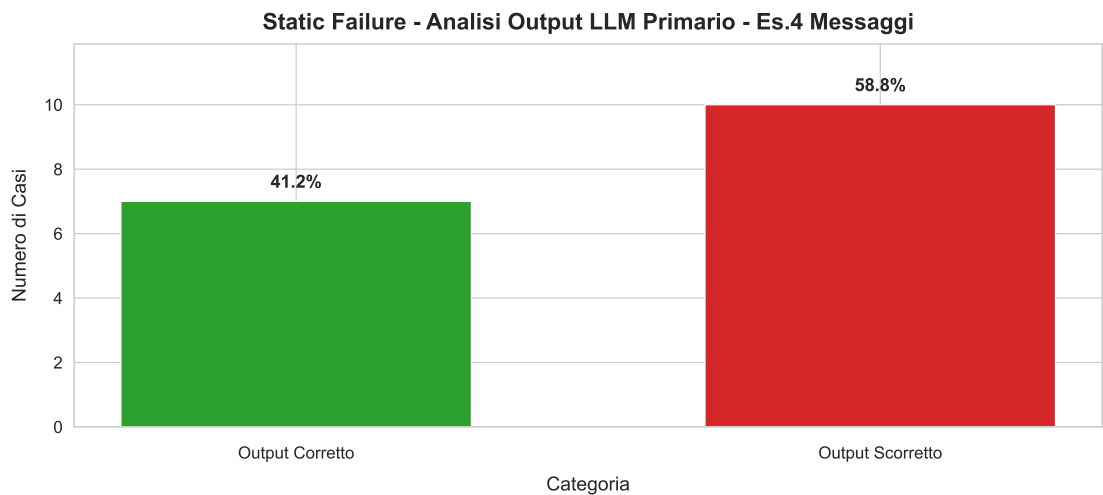


Figura 3.31: Risultati analisi con LLM per output, categoria: Static failure - esercitazione 4.

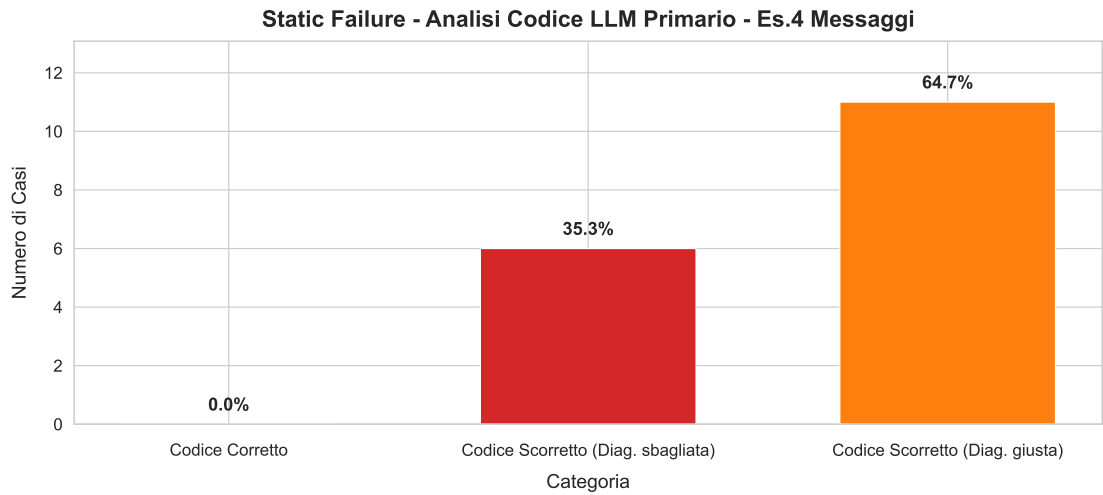


Figura 3.32: Risultati analisi con LLM per codice, categoria: Static failure - esercitazione 4.

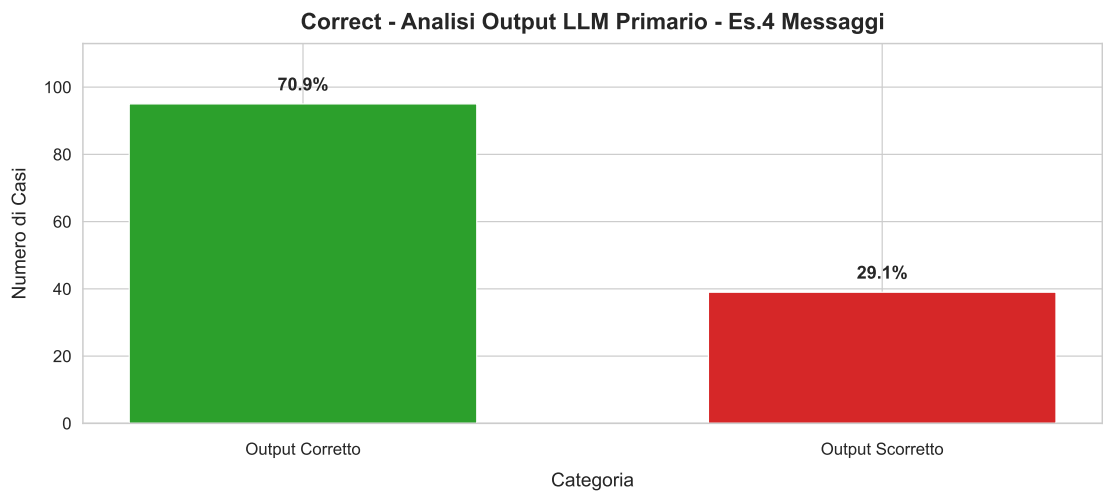


Figura 3.33: Risultati analisi con LLM per output, categoria: Correct - esercitazione 4.

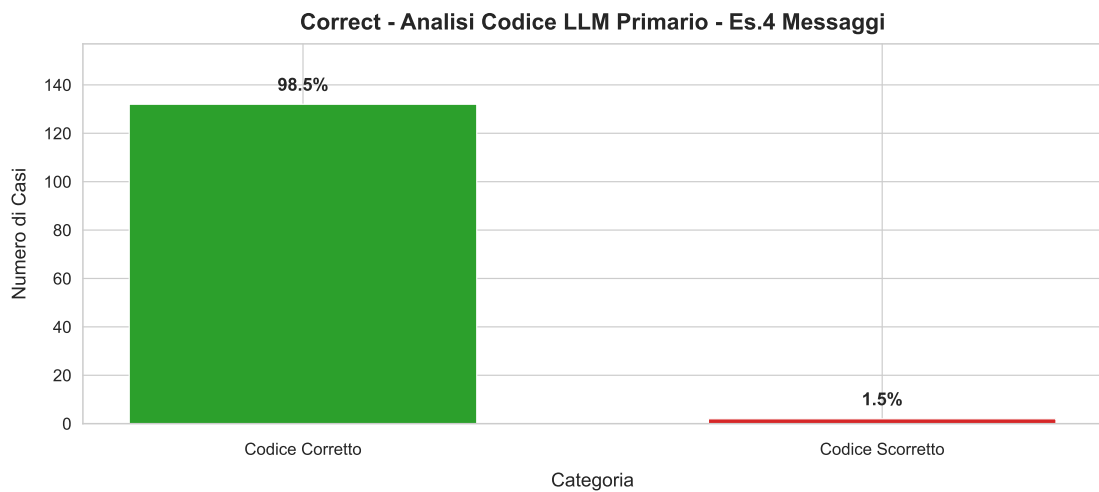


Figura 3.34: Risultati analisi con LLM per codice, categoria: Correct - esercitazione 4.

Descrizione risultati

Le categorie **timeout** e **ipc_leak** mostrano un'accuratezza perfetta sia nella valutazione che nella diagnosi, confermando il trend delle precedenti esercitazioni. I casi di **dynamic_failure** presentano invece un notevole divario tra valutazione piuttosto accurata (76,5% sull'output e 88,2% sul codice) e diagnosi meno affidabili sull'output, scendendo al 60%, ma comunque molto affidabili sul codice(97,3%). Gli **static_failure** presentano valutazione dell'output molto modesta (41,2%), mentre quella del codice è perfetta, ma con diagnosi non sempre credibili. Infine i programmi classificati come **correct** presentano valutazione dell'output piuttosto buona (70,9%), e valutazione del codice molto accurata, con relative diagnosi totalmente affidabili.

Failure category	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
crash	N/A	N/A	N/A	N/A
timeout	N/A	100.0% (55/55)	N/A	100.0% (55/55)
ipc_leak	N/A	100.0% (21/21)	N/A	100.0% (21/21)
dynamic_failure	76.5% (65/85)	88.2% (75/85)	60.0% (39/65)	97.3% (73/75)
static_failure	41.2% (7/17)	100.0% (17/17)	100.0% (7/7)	64.7% (11/17)
correct	70.9% (95/134)	98.5% (132/134)	100.0% (95/95)	100.0% (132/132)

Tabella 3.11: Accuratezza per categoria di fallimento

Per quanto riguarda gli esercizi specifici, emergono differenze significative. **Grafo delle dipendenze** mostra performance molto elevate sia nella valutazione sia nella diagnosi basata sul codice, mentre la diagnosi sull'output rimane più debole. **Load balancing** evidenzia qualche difficoltà in più nella valutazione dell'output (68,0%), a fronte di analisi del codice particolarmente accurata e diagnosi spesso affidabili. **Registro distribuito** presenta una valutazione dell'output ottima ma una diagnosi sull'output molto bassa (18,8%), evidenziando difficoltà nell'individuare il reale problema presente all'interno dell'output. **Server sincroni multipli** allo stesso modo presenta diagnosi output debole, mentre l'analisi del codice risulta piuttosto affidabile.

Exercise	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
grafo_delle_dipendenze	94.1% (95/101)	94.6% (105/111)	60.0% (6/10)	97.4% (37/38)
load_balancing	68.0% (68/100)	100.0% (104/104)	96.2% (25/26)	86.5% (32/37)
registro_distribuito	89.8% (88/98)	99.0% (100/101)	18.8% (3/16)	100.0% (39/39)
server_sincroni_multipli	80.4% (78/97)	95.5% (107/112)	38.5% (5/13)	96.3% (52/54)

Tabella 3.12: Accuratezza per esercizio specifico

3.3.5 Risultati esercitazione 5 - Server Multithread

Le principali statistiche relative all'esercitazione sono le seguenti:

- Numero di studenti con almeno 1 commit: 110
- Numero totale di commit git: 1040
- Numero totale di modifiche agli esercizi: 877
- Numero di modifiche analizzate tramite LLM: 500

In relazione alla distribuzione globale, sono state raccolte le seguenti statistiche per ciascun esercizio dell'esercitazione:

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct
remote_procedure_call	105	0	46	4	23	17	76
server_aggregatore_thread	7	0	23	1	162	2	49
un_primo_esempio_di_server_multithread	105	0	36	2	45	41	133

Tabella 3.13: Conteggio delle categorie di fallimento per ciascun esercizio

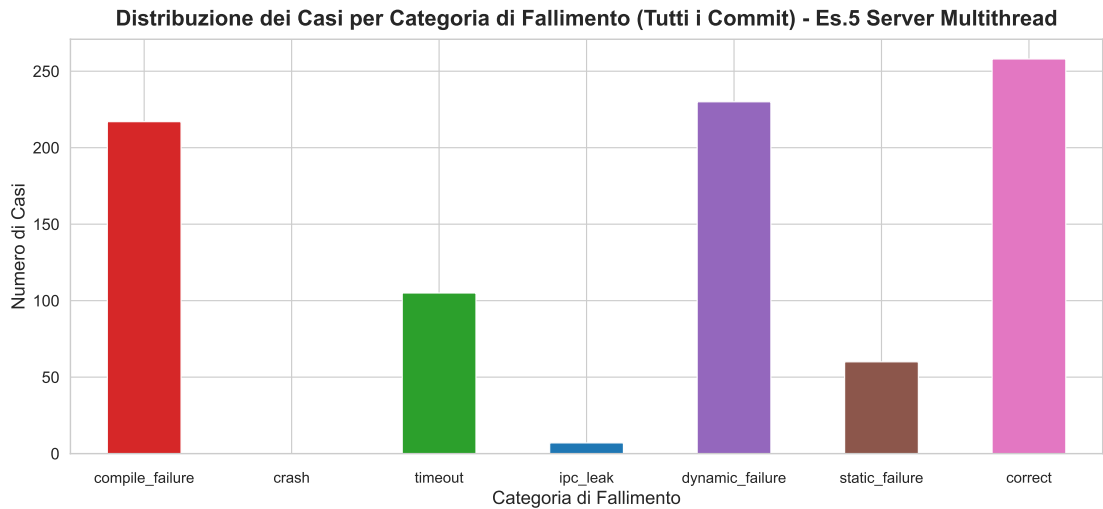


Figura 3.35: Distribuzione globale delle categorie di fallimento per esercitazione 5

Risultati Analisi mediante LLM

La distribuzione dei commit analizzati tramite LLM, selezionati in proporzione alla distribuzione completa, è riportata di seguito:

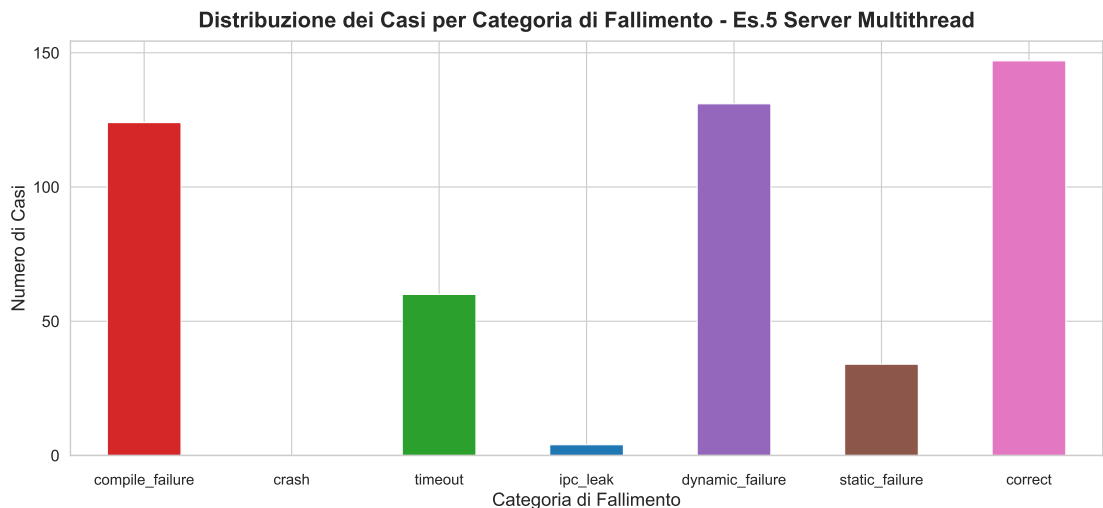


Figura 3.36: Distribuzione dei fallimenti nei commit analizzati con LLM – esercitazione 5

I risultati prodotti dagli LLM sono stati analizzati con riferimento

alle categorie **dynamic_failure**, **static_failure** e **correct**, e sono rappresentati nei grafici riportati di seguito.

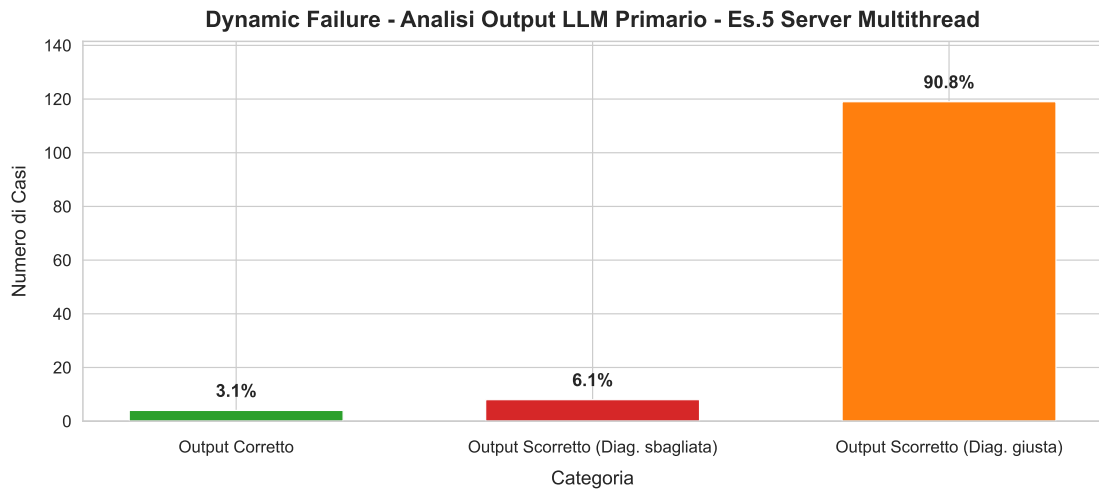


Figura 3.37: Risultati analisi con LLM per output, categoria: Dynamic failure - esercitazione 5.

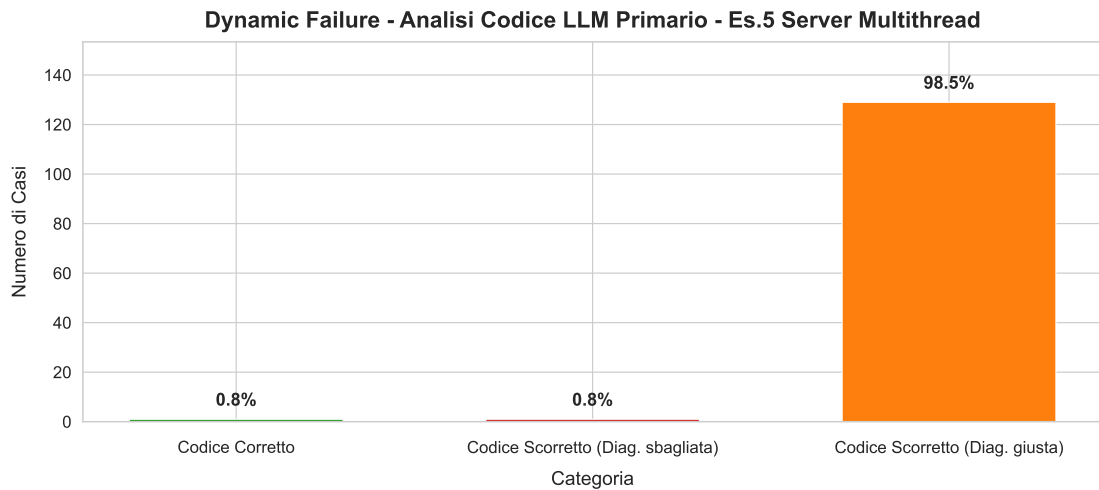


Figura 3.38: Risultati analisi con LLM per codice, categoria: Dynamic failure - esercitazione 5.

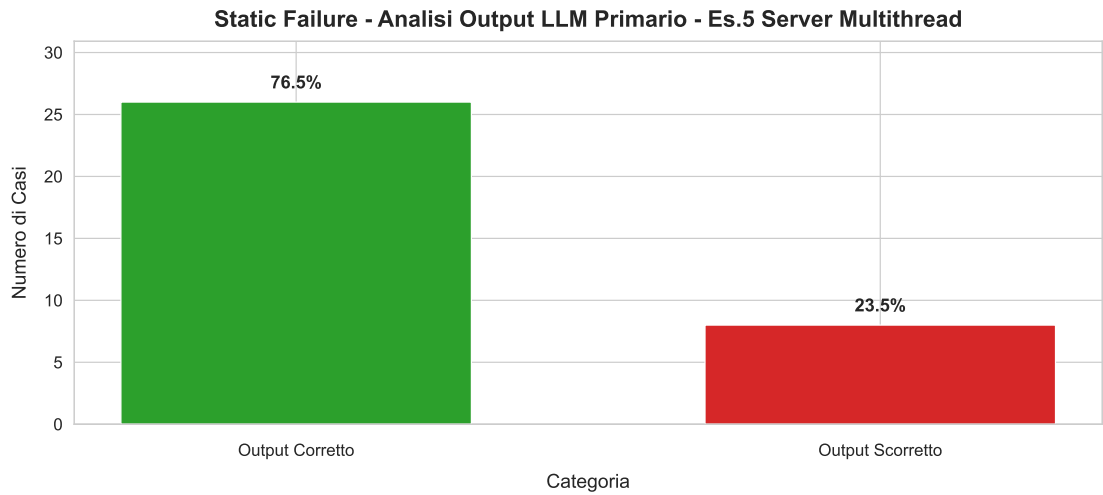


Figura 3.39: Risultati analisi con LLM per output, categoria: Static failure - esercitazione 5.

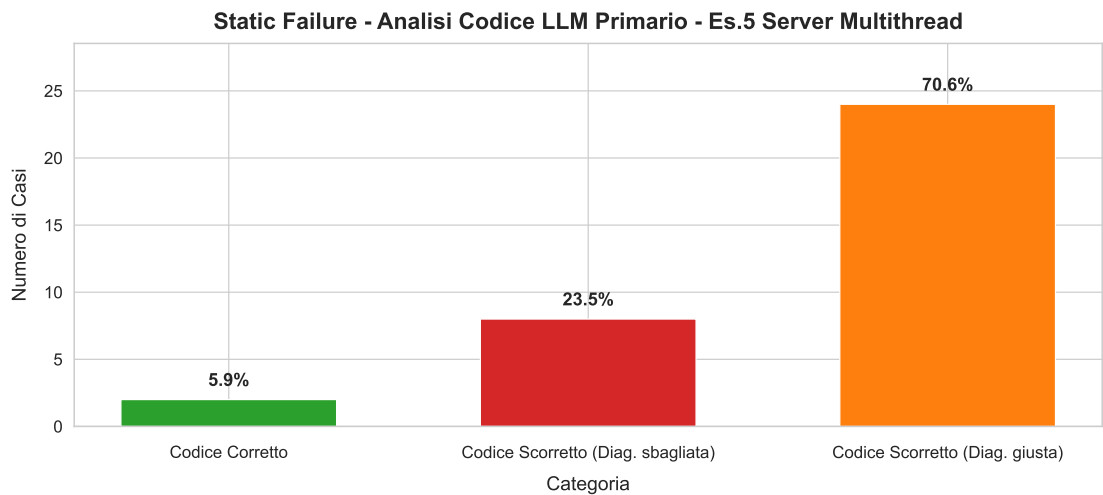


Figura 3.40: Risultati analisi con LLM per codice, categoria: Static failure - esercitazione 5.

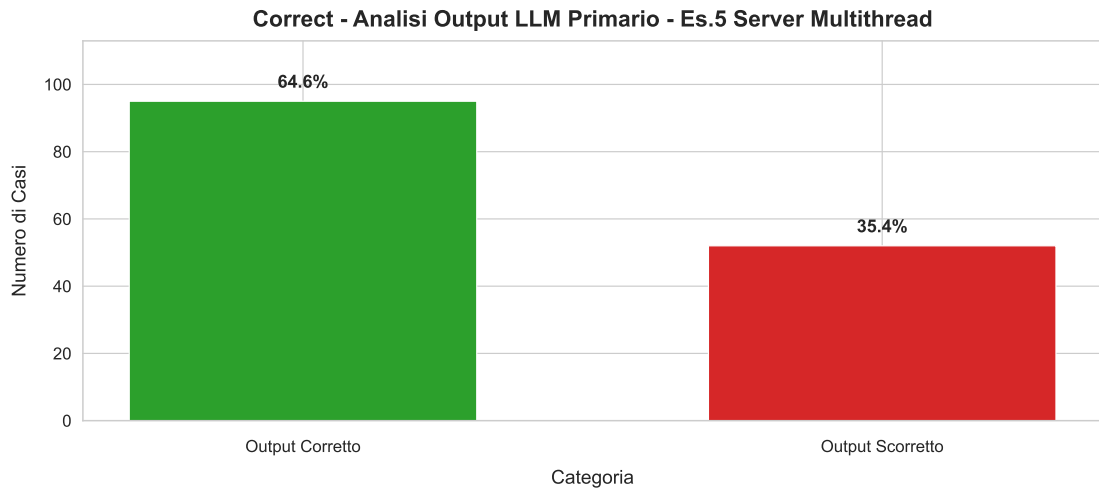


Figura 3.41: Risultati analisi con LLM per output, categoria: Correct - esercitazione 5.

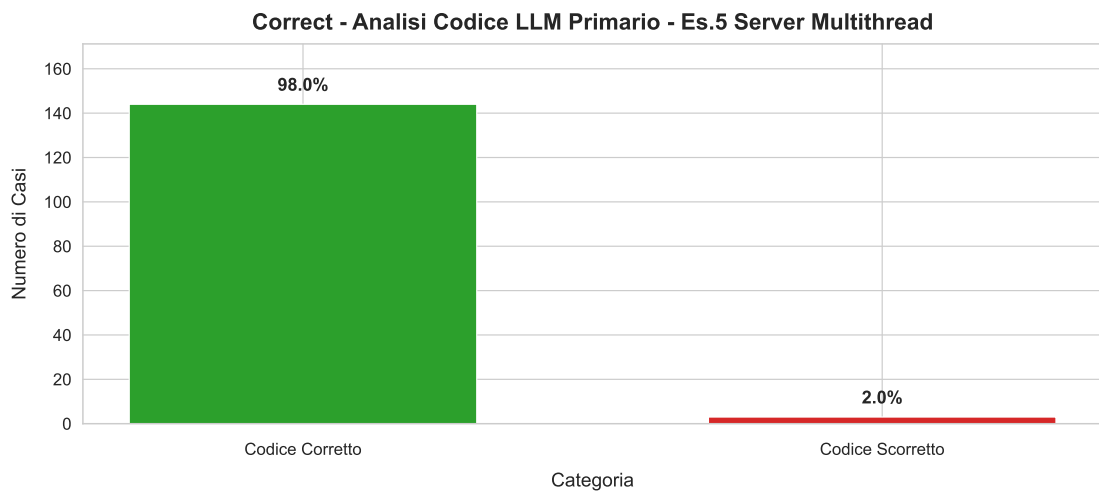


Figura 3.42: Risultati analisi con LLM per codice, categoria: Correct - esercitazione 5.

Descrizione risultati

Le categorie **timeout** e **ipc_leak** mostrano un'accuratezza pressoché perfetta sia nella valutazione che nella diagnosi, confermando il trend osservato in ogni esercitazione. I casi di **dynamic_failure** presentano performance molto elevate sia nella valutazione (96,9% e 99,2%)

che nella diagnosi, con un leggero calo nell'analisi dell'output (93,7%), indicando che il comportamento dinamico viene tipicamente compreso, ma con qualche incertezza residua. Gli **static_failure** mostrano ottime capacità di valutazione sia dell'output che del codice, evidenziando però per quest'ultimo diagnosi non sempre attendibili, con il 75%. Infine i programmi classificati come **correct** presentano valutazione dell'output discreta (64,6%), e valutazione del codice con percentuale molto elevata (98%), ma relative diagnosi perfettamente accettabili.

Failure category	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
crash	N/A	N/A	N/A	N/A
timeout	N/A	100.0% (60/60)	N/A	95.0% (57/60)
ipc_leak	N/A	100.0% (4/4)	N/A	100.0% (4/4)
dynamic_failure	96.9% (127/131)	99.2% (130/131)	93.7% (119/127)	99.2% (129/130)
static_failure	76.5% (26/34)	94.1% (32/34)	100.0% (26/26)	75.0% (24/32)
correct	64.6% (95/147)	98.0% (144/147)	100.0% (95/95)	100.0% (144/144)

Tabella 3.14: Accuratezza per categoria di fallimento

Per quanto riguarda gli esercizi specifici, **Remote procedure call** mostra una valutazione dell'output discreta, ma un'ottima analisi del codice e diagnosi solide. **Server aggregatore thread** presenta ottime performance, mostrando diagnosi eccellenti e valutazione del codice perfetta, evidenziando leggermente più difficoltà nell'interpretazione dell'output da parte del LLM. **Un primo esempio di server multithread** è anch'esso caratterizzato da performance equilibrate: valutazione dell'output molto alta e ottima analisi del codice, anche se le diagnosi risultano leggermente meno accurate.

Exercise	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
remote_procedure_call	75.0% (102/136)	98.7% (151/153)	63.6% (7/11)	92.3% (48/52)
server_aggregatore_thread	83.0% (112/135)	100.0% (143/143)	100.0% (96/96)	98.3% (113/115)
un_primo_esempio_di_server_multithread	85.3% (157/184)	98.0% (192/196)	80.0% (16/20)	89.8% (53/59)

Tabella 3.15: Accuratezza per esercizio specifico

3.4 Risultati dell’analisi replicata mediante modello GPT-OSS-20b

L’intera analisi precedentemente descritta è stata replicata utilizzando il modello *GPT-OSS-20b*, ospitato e interrogato tramite un server remoto accessibile attraverso la **VPN** universitaria. Si è scelto di analizzare gli stessi commit per garantire una comparazione diretta e non ambigua tra le prestazioni dei modelli, raccogliendo i risultati relativi a ciascuna esercitazione secondo la medesima procedura.

3.4.1 Differenze tra modello GPT-4o e GPT-OSS-20b

GPT-OSS-20b è un modello *open-weight*: OpenAI ne ha pubblicato i pesi neurali sotto licenza *Apache 2.0*, consentendone l’esecuzione su hardware privato, come avvenuto anche per la presente analisi. Al contrario, *GPT-4o* è un modello proprietario, accessibile esclusivamente tramite servizi cloud quali OpenAI e Azure. Un’ulteriore

differenza rilevante, ma ininfluyente in questo studio, riguarda gli *input* supportati:

- *GPT-4o* è un modello multimodale, in grado di elaborare testo, immagini e audio.
- *GPT-OSS-20b*, invece, accetta unicamente input testuali.

In particolare, *GPT-OSS-20b* può essere ottimizzato con notevole efficacia per compiti di *reasoning* e *coding*, risultando particolarmente adatto a contesti in cui è richiesto un controllo diretto sia sul modello sia sull'infrastruttura di esecuzione, garantendo maggiore flessibilità in termini di deployment e adattamento a compiti specifici.[11]

3.4.2 Risultati aggregati modello GPT-OSS-20b e differenze con GPT-4o

Dalle analisi effettuate è possibile osservare prestazioni simili nella corretta valutazione dell'output, mentre si denota una minor efficacia nell'individuare casi di *dynamic failure*. Tale difficoltà può essere ricondotta alla complessità intrinseca dell'interpretazione del comportamento concorrente basata esclusivamente sugli output di esecuzione, soprattutto quando gli errori dipendono da fenomeni di sincronizzazione tra processi e/o thread. Per quanto riguarda l'analisi del codice, si osservano prestazioni elevate nelle categorie *crash* e *timeout* nella valutazione di codice errato, segno di una buona capacità nel rico-

noscere errori concorrenti evidenti o facilmente rilevabili. Le diagnosi, tuttavia, risultano meno accurate rispetto al modello proprietario. Un andamento simile si riscontra anche nelle restanti categorie, dove l'accuratezza è generalmente inferiore sia nelle valutazioni che nelle diagnosi. In particolare, per gli *static failure* si nota un divario significativo: la valutazione del codice errato è complessivamente buona, ma accompagnata da diagnosi imprecise, fenomeno osservato anche per l'altro modello. Nei casi *correct*, invece, avviene l'opposto: le diagnosi sono accurate quando il codice viene giudicato corretto (compito relativamente semplice), ma si riscontra una tendenza ad individuare errori inesistenti, con conseguenti valutazioni inaccurate.

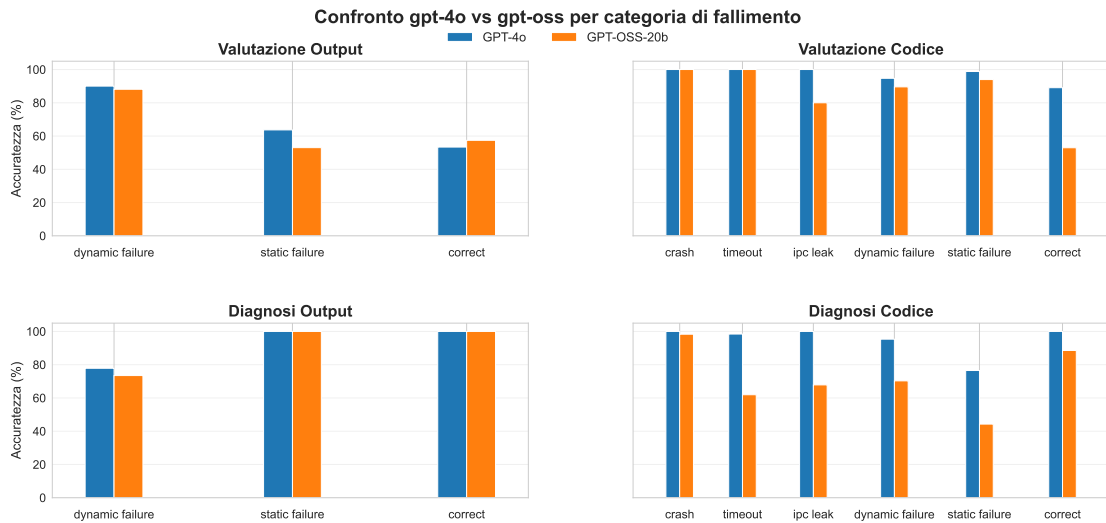


Figura 3.43: Accuratezza dei risultati generati dal modello GPT-OSS-20b, suddivisi per categoria di fallimento, a confronto con risultati generati dal modello GPT-4o

Per quanto riguarda l'analisi dei risultati suddivisi per esercitazione, il modello mostra una buona capacità di valutazione dell'output, risul-

tando persino più accurato di *GPT-4o* nelle prime due esercitazioni. Le diagnosi, tuttavia, tendono ad essere spesso imprecise, in particolare nell'esercitazione 2 relativa ai *monitor*, sebbene le prestazioni complessive dei due modelli rimangano relativamente simili. L'analisi del codice evidenzia invece prestazioni inferiori in tutte le esercitazioni, sia nella valutazione che, soprattutto, nelle diagnosi, confermando una maggiore difficoltà proprio nella seconda esercitazione. Nel complesso, emerge una significativa inaccuracy nell'analisi statica dei programmi, con una tendenza a fornire diagnosi meno affidabili rispetto al modello principale.

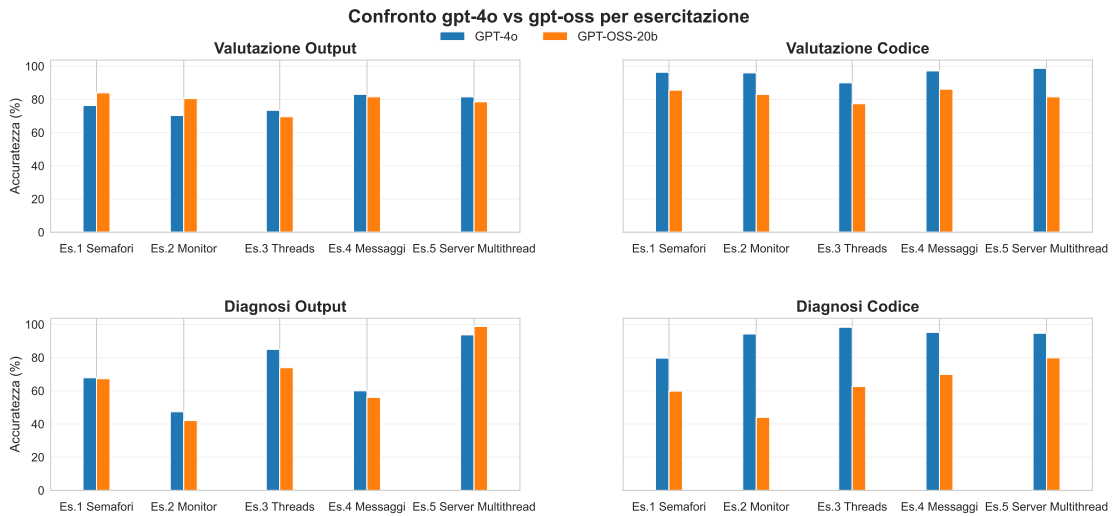


Figura 3.44: Accuratezza dei risultati generati dal modello GPT-OSS-20b, suddivisi per esercitazione, a confronto con risultati generati dal modello GPT-4o

Considerando una visione complessiva dei risultati ottenuti dai due modelli, è possibile trarre alcune conclusioni definitive riguardo all'accuratezza nella valutazione dei programmi concorrenti, includendo le diverse esercitazioni e le varie categorie di fallimento. Per ciò che

concerne l'analisi dell'*output*, si osserva una maggior accuratezza nelle valutazioni da parte di *GPT-OSS-20b*, sebbene accompagnata da diagnosi leggermente meno precise. Nel complesso, le prestazioni dei due modelli risultano comunque comparabili, con valori compresi tra il 76% e l'80%. L'analisi del codice mostra invece una situazione diversa, infatti *GPT-4o* raggiunge un'accuratezza nelle valutazioni più elevata, mentre *GPT-OSS-20b* mantiene comunque un buon livello, pari all'83,2%. Le differenze più evidenti emergono però nelle diagnosi basate sul codice, dove *GPT-4o* ottiene risultati nettamente superiori (92,9%), a fronte di una percentuale notevolmente più bassa per *GPT-OSS-20b* (65%), evidenziando diagnosi spesso distanti rispetto alla vera natura del problema.

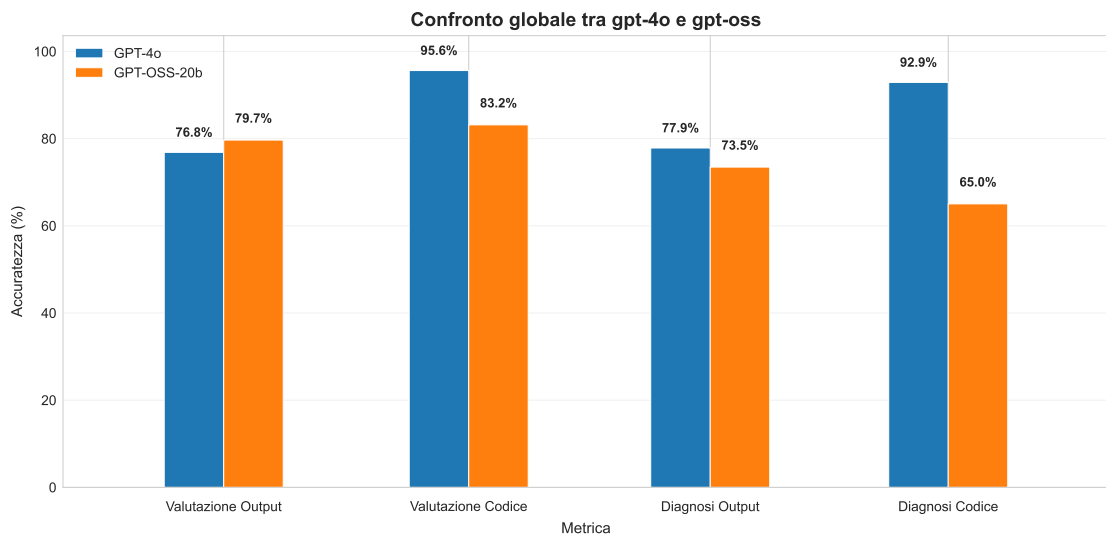


Figura 3.45: Accuratezza dei risultati generati dal modello GPT-OSS-20b a confronto con risultati generati dal modello GPT-4o

Le divergenze osservate possono essere attribuite alle diverse caratteristiche dei due modelli. *GPT-4o*, essendo un modello proprietario di

dimensioni maggiori e addestrato su un insieme più ampio e diversificato di esempi di codice, dispone di rappresentazioni interne più ricche delle strutture sintattiche e semantiche dei programmi. Questo gli consente di individuare con maggior efficacia anche errori concorrenti particolarmente complessi, tipici dei programmi analizzati. *GPT-OSS-20b*, pur mostrando prestazioni peggiori nell'analisi statica del codice, permette di ottenere risultati migliori nella valutazione dell'output, considerando che tale processo si basa sull'interpretazione dei risultati di esecuzione piuttosto che sulla comprensione approfondita della struttura interna del programma.

3.5 Analisi statica mediante Semgrep e regole *Oxdea*

In seguito all'analisi definitiva condotta tramite LLM, si è deciso di estendere la valutazione dei programmi anche all'analisi statica mediante l'impiego di regole generiche, in contrasto con quelle personalizzate utilizzate nel ciclo di studio. L'obiettivo è stato quello di individuare i **warning** più ricorrenti nelle diverse esercitazioni ed esercizi, associandoli alle rispettive categorie di fallimento. Tra i vari strumenti disponibili, la scelta è ricaduta nuovamente su **Semgrep**, in particolare sulle regole sviluppate da **Oxdea** [7], che hanno prodotto un numero significativo di warning, a differenza delle regole standard. Tali regole

coprono diverse categorie di vulnerabilità e sono progettate per individuare pattern potenzialmente rischiosi in codice C. È opportuno menzionare anche il tool **SonarQube**, preso in considerazione durante la fase di analisi. Tuttavia, le limitazioni imposte dalla versione *Community* non hanno consentito di effettuare le valutazioni necessarie, rendendo lo strumento inadatto agli obiettivi prefissati, pur rimanendo potenzialmente applicabile nella sua versione *Developer*.

Di seguito sono presentati i risultati dell'analisi:

- Conteggio del numero di warning per esercitazione, esercizio e failure category.
- Calcolo top-10 regole di analisi che hanno sollevato warning e relativo conteggio.

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct	Total-Warnings
calcolo_parallelo_su_un_vettore_condiviso	1287	28	9	54	403	655	1258	3694
coppia_di_buffer	1232	0	203	90	176	439	1104	3244
lettori-scrittori_con_semafori	1493	0	98	317	656	614	1592	4770
su_una_coppia_di_valori_condivisa								
simulazione_di_un_disco_con_un_vettore_circolare	538	0	214	83	525	0	863	2223

Tabella 3.16: Conteggio dei warning segnalati per ciascun esercizio e per ogni categoria di fallimento - esercitazione 1

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct	Total-Warnings
lettori-scrittori_con_monitor_e_processi	1764	0	240	60	36	36	1844	3980
prelievi_multipli	1056	24	138	24	88	352	712	2394
produttore-consumatore_con_priorita	1485	44	231	0	208	275	1210	3453
simulazione_di_un_disco_con_un_vettore_circolare	816	6	116	0	122	0	618	1678

Tabella 3.17: Conteggio dei warning segnalati per ciascun esercizio e per ogni categoria di fallimento - esercitazione 2

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct	Total-Warnings
lettori-scrittori_su_piu_oggetti_monitor	68	963	277	0	40	444	1836	3628
produttore-consumatore_con_priorita	1320	42	169	0	661	233	1020	3445
una_struttura_dati_stack_thread-safe	1520	117	226	0	379	156	1183	3581

Tabella 3.18: Conteggio dei warning segnalati per ciascun esercizio e per ogni categoria di fallimento - esercitazione 3

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct	Total-Warnings
grafo_delle_dipendenze	2213	0	1151	361	1044	30	1735	6534
load_balancing	6	0	12	27	147	27	224	443
registro_distribuito	675	0	351	30	293	141	822	2312
server_sincroni_multipli	456	0	400	240	766	170	830	2862

Tabella 3.19: Conteggio dei warning segnalati per ciascun esercizio e per ogni categoria di fallimento - esercitazione 4

Exercise	Compile failure	Crash	Timeout	IPC Leak	Dynamic failure	Static failure	Correct	Total-Warnings
remote_procedure_call	2526	0	1239	105	602	440	2045	6957
server_aggregatore_thread	91	0	352	16	1549	32	797	2837
un_primo_esempio_di_server_multithread	949	0	436	25	526	466	1598	4000

Tabella 3.20: Conteggio dei warning segnalati per ciascun esercizio e per ogni categoria di fallimento - esercitazione 5

Posizione	Regola	Warning-Count
1	interesting-api-calls	30093
2	insecure-api-rand-srand	16294
3	missing-return	11879
4	unchecked-ret-malloc	2028
5	mismatched-memory-management	1256
6	typos	429
7	format-string-bugs	39
8	incorrect-use-of-sizeof	9
9	signed-unsigned-conversion	7
10	incorrect-use-of-free	1

Tabella 3.21: Top-10 regole di analisi statica che hanno sollevato warning

Effetti dell'applicazione delle regole 0xdea

Durante l'applicazione delle regole al codice delle esercitazioni, è emerso che alcuni **warning** sono "**fisiologici**", cioè derivano direttamente dalle caratteristiche strutturali degli esercizi. Ad esempio alcune regole classificano l'uso di **primitive di sincronizzazione** (semafori, mutex, shared memory) come potenzialmente pericolose o sensibili, attivando la regola *interesting-api-calls* praticamente in ogni esercitazione, data la programmazione concorrente. Inoltre alcune funzioni o costrutti sono richiesti esplicitamente per completare gli esercizi, e quindi generano warning indipendentemente dalla qualità del codice dello studente. Un secondo insieme di warning segnalati erroneamente deriva dalle **semplificazioni didattiche** introdotte per agevolare gli studenti, come:

- assenza di controlli sul valore restituito da *malloc()*
- assenza di verifiche su *pthread_create()* o altre *API* di sistema
- gestione semplificata degli errori

Poiché tali controlli non sono richiesti, le regole *0xdea* li segnalano come potenziali vulnerabilità, pur non rappresentando errori reali nel contesto didattico. Per distinguere i warning effettivamente imputabili allo studente da quelli intrinseci all'esercizio, è stata applicata una procedura di **normalizzazione**, applicando le regole *0xdea* al codice dello studente per ogni commit e successivamente anche al codice della soluzione ufficiale dell'esercitazione. È stata quindi effettuata una **sottrazione** dei warning fisiologici, calcolando per ogni categoria di regole la differenza tra i warning generati dal codice dello studente e quelli prodotti dalla soluzione ufficiale dell'esercitazione:

$$\text{differenza} = \max(0, \text{warningstudente} - \text{warningsoluzione})$$

La differenza viene forzata a zero quando lo studente produce meno warning della soluzione (tipicamente perché il codice è incompleto). Se in un commit compaiono più warning della stessa categoria, esso viene comunque conteggiato una sola volta. Questa procedura permette di isolare i warning che rappresentano errori reali, distinguendoli da quelli dovuti alla struttura dell'esercizio o alle semplificazioni didattiche. In questo modo è possibile:

- Determinare quali categorie di vulnerabilità sono più problematiche per gli studenti.
- Distinguere errori reali da warning strutturali.
- Confrontare in modo omogeneo esercitazioni con caratteristiche differenti.

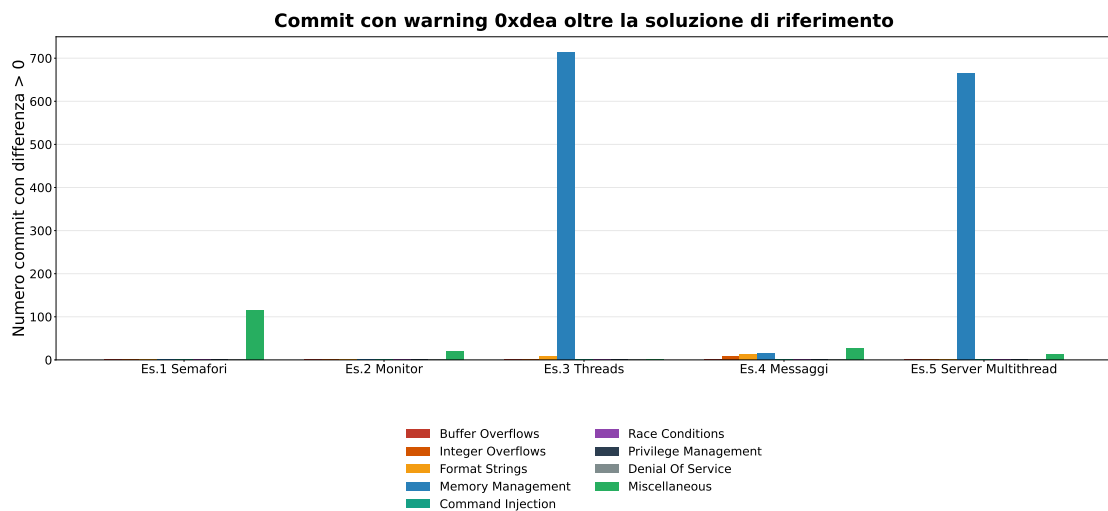


Figura 3.46: Numero commit con warning Semgrep-Oxdea oltre la soluzione di riferimento

I medesimi risultati possono essere descritti attraverso un'altra versione del grafico, riportando la percentuale sul totale di commit analizzati, anziché il numero assoluto:

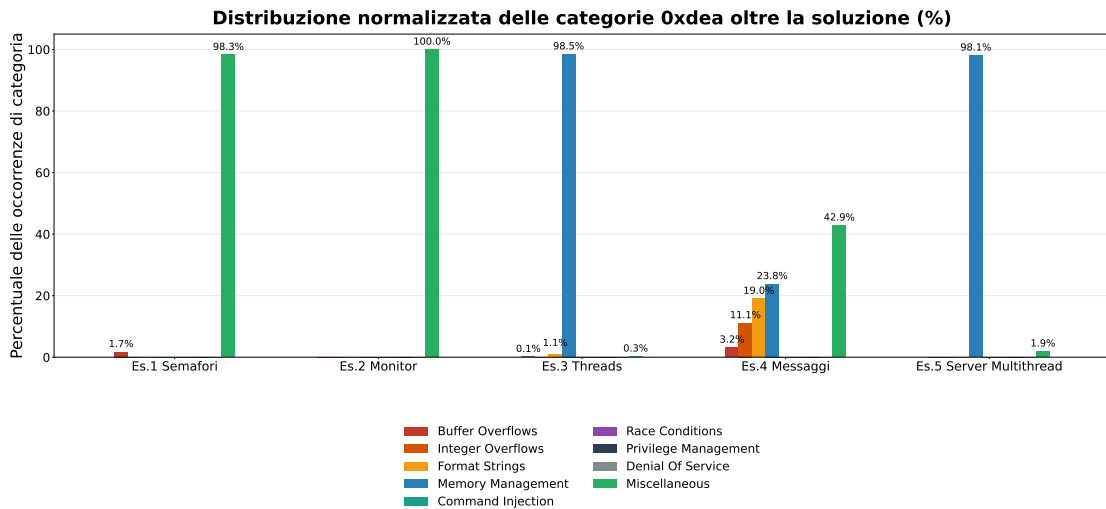


Figura 3.47: Percentuale normalizzata dei commit con warning Semgrep-Oxdea oltre la soluzione di riferimento

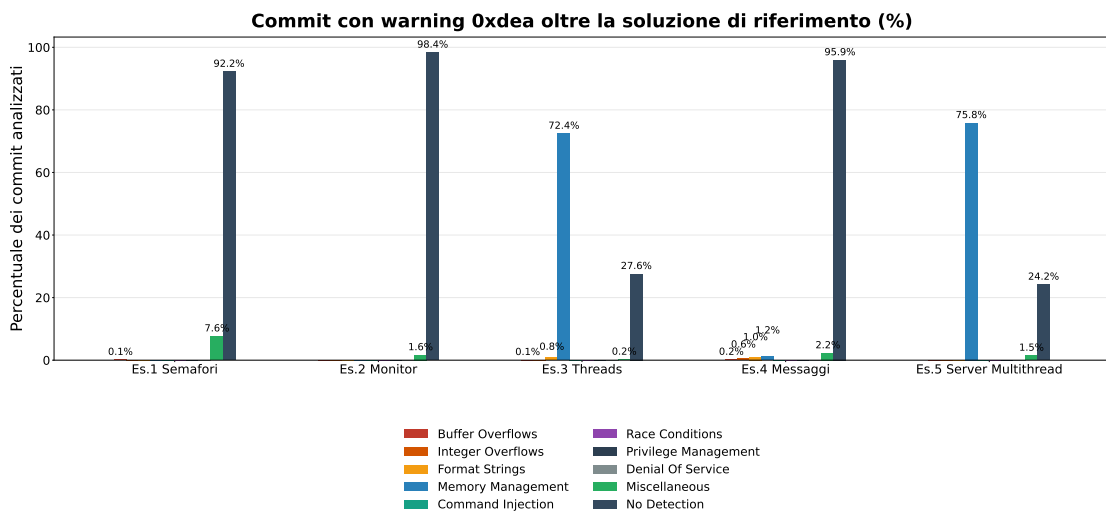


Figura 3.48: Percentuale dei commit con warning Semgrep-Oxdea oltre la soluzione di riferimento

Infine, per analizzare la **sovrapposizione** tra Semgrep e le altre analisi, sono stati selezionati i commit classificati come **correct** e **static_failure** e, su questo insieme, sono state calcolate quattro percentuali distinte: commit segnalati esclusivamente da Semgrep, commit segnalati esclusivamente dalle regole custom dei nostri script, commit

segnalati da entrambi gli strumenti e commit non segnalati da nessuno dei due.

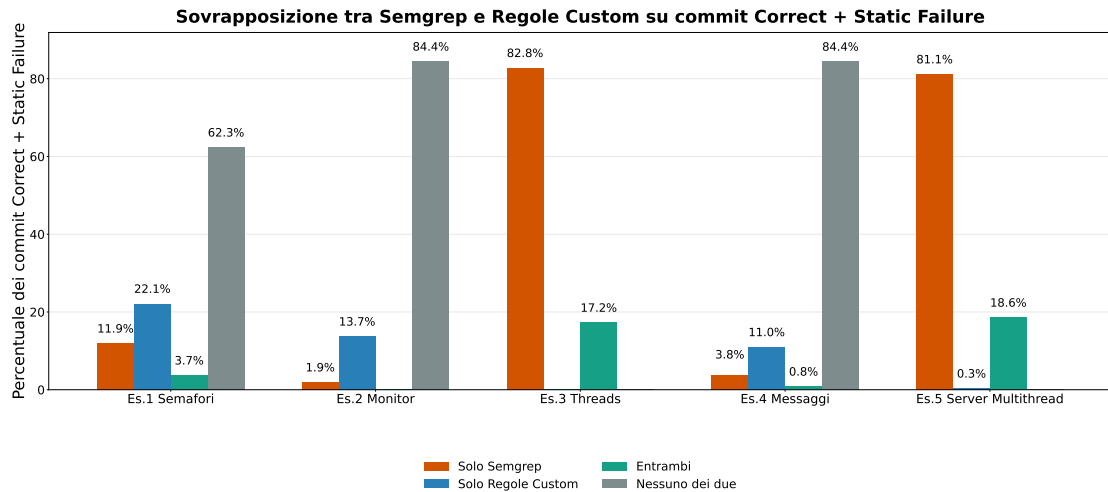


Figura 3.49: Sovrapposizione tra warning generati da Semgrep con regole Oxdea e warning segnalati dalle regole custom utilizzate per l'analisi principale

Capitolo 4

Conclusioni

La valutazione finale, che considera nel complesso tutte le esercitazioni analizzate mediante il modello **GPT-4o**, restituisce un quadro piuttosto chiaro, mettendo in evidenza l'eccellente capacità nel riconoscimento di failure di natura *sistemica*, ambito nel quale le performance risultano costantemente elevate. Le categorie **crash**, **timeout** e **IPC leak** raggiungono infatti un'accuratezza del 100% per la valutazione, ma con diagnosi leggermente meno accurate per i casi di **timeout**. Ciò suggerisce che, quando la natura del fallimento è immediatamente evidente e non richiede inferenze sul comportamento del programma, il modello offre ottime prestazioni. La situazione diventa sensibilmente più complessa quando si considerano fallimenti di natura *logica*, che richiedono un'analisi più profonda del comportamento del programma. I **dynamic failure** mostrano una valutazione molto precisa (oltre il 90% sia su output che su codice), ma con diagnosi sull'output che cala

al 77,9%. Quanto accade per l'analisi degli **static failure** è piuttosto interessante, osservando diagnosi basate sul codice meno accurate tra tutte le categorie (76,5%). Ciò suggerisce una maggior difficoltà del LLM nel ragionare su proprietà statiche del codice, come condizioni di sincronizzazione, assenza di race condition o correttezza strutturale, rispetto ad errori più evidenti che causano la generazione di output errato, evidenziando miglior capacità del modello nel gestire i fallimenti dinamici rispetto a quelli statici. Gli static failure rappresentano infatti la categoria più complessa da analizzare, con accuratezza rispetto alla valutazione dell'output corretto piuttosto bassa (63,7%), ma prestazioni ottimali (98,8%) per la valutazione di codice errato. Tale divario è il più evidente tra tutte le categorie e conferma la maggior efficacia del LLM nell'analisi diretta del codice rispetto all'analisi dell'output generato, che è certamente meno strutturato ed induce il modello a considerarlo errato anche quando non lo è. Quest'ultimo risultato può essere osservato allo stesso modo anche per i casi **correct**, dimostrando complessità nell'affermare la correttezza dell'output generato (53,4%), ricercando errori di sincronizzazione o comunque tipici di programmi concorrenti come descritto in precedenza.

Failure category	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
crash	N/A	100.0% (80/80)	N/A	100.0% (80/80)
timeout	N/A	100.0% (191/191)	N/A	98.4% (188/191)
ipc_leak	N/A	100.0% (43/43)	N/A	100.0% (43/43)
dynamic_failure	90.0% (307/341)	94.7% (323/341)	77.9% (239/307)	95.4% (308/323)
static_failure	63.7% (107/168)	98.8% (166/168)	100.0% (107/107)	76.5% (127/166)
correct	53.4% (420/787)	89.1% (701/787)	100.0% (420/420)	100.0% (701/701)

Tabella 4.1: Accuratezza globale per categoria di fallimento

Osservando le prestazioni suddivise per tipologia di esercitazione, emergono differenze significative tra i vari paradigmi di concorrenza. Le esercitazioni basate su **semafori** e **monitor** (Es.1 ed Es.2) mostrano una discreta accuratezza nella valutazione dell'output (76,3% e 70,3%), con prestazioni migliori relativamente al codice (96,4% e 96,0%), ma evidenziano un netto calo nella *diagnosi dell'output*, soprattutto per i monitor, che raggiungono solo il 47,4%, indicando maggiore difficoltà nell'interpretare meccanismi di sincronizzazione. Le esercitazioni basate su **threads** (Es.3) presentano invece un comportamento più bilanciato: la valutazione dell'output si attesta al 73,5%, mentre la valutazione del codice raggiunge il 90%. Anche le metriche di diagnosi risultano piuttosto solide (85% per l'output e 98,3% per il codice), suggerendo una migliore capacità del modello nel gestire scenari concorrenti più espliciti e strutturati. Relativamente allo scambio di **messaggi** (Es.4), si osserva un'elevata accuratezza nella valutazione del codice (97,2%) e buone performance sull'output (83,1%), ma con diagnosi molto meno stabili (60%). Questo comportamento conferma che in assenza di meccanismi di sincronizzazione condivisa, l'identifi-

cazione delle cause di fallimento risulti più complessa e meno deterministica. Infine, gli esercizi relativi a **server multithread** (Es.5) rappresentano il caso più efficace per il modello, con prestazioni complessivamente elevate e particolarmente rilevanti nella *diagnosi dell'output*, che raggiunge il valore più elevato, con il 93,7% di accuratezza, insieme a buoni risultati anche nella valutazione del codice (81,5%). Tali risultati suggeriscono una maggiore affidabilità del modello rispetto all'interpretazione di scenari architetetturalmente più regolari e ben strutturati.

Tipologia di esercitazione	Valutazione Output	Valutazione Codice	Diagnosi Output	Diagnosi Codice
Es.1 Semafori	76.3% (355/465)	96.4% (482/500)	67.9% (38/56)	79.7% (114/143)
Es.2 Monitor	70.3% (327/465)	96.0% (480/500)	47.4% (9/19)	94.3% (82/87)
Es.3 Threads	73.5% (291/396)	90.0% (450/500)	85.0% (34/40)	98.3% (176/179)
Es.4 Messaggi	83.1% (329/396)	97.2% (416/428)	60.0% (39/65)	95.2% (160/168)
Es.5 Server Multithread	81.5% (371/455)	98.8% (486/492)	93.7% (119/127)	94.7% (214/226)

Tabella 4.2: Accuratezza globale per tipologia di esercitazione

Nel complesso, l'analisi mette in evidenza un pattern piuttosto stabile nelle capacità del modello: l'LLM mostra infatti un'elevata efficacia nell'analisi statica del codice, riuscendo a distinguere con buona accuratezza il codice corretto da quello errato, e nel riconoscimento di failure di natura *sistemica*, come *crash*, *timeout* e *IPC leak*, mentre presenta maggiori difficoltà nella valutazione dell'output generato e, soprattutto, nella formulazione delle relative diagnosi. Il divario descritto diventa particolarmente evidente nei casi che richiedono un ragionamento più profondo sul comportamento del programma, so-

prattutto nel momento in cui bisogna valutare problemi di sincronizzazione, concorrenza complessa o proprietà statiche del programma, ambiti nei quali l'analisi non può basarsi esclusivamente su segnali evidenti o strutturali del codice. In particolare, i risultati relativi agli *static failure* confermano tale limite: nonostante l'elevata accuratezza rispetto alla classificazione del codice errato (98,8%), le rispettive diagnosi risultano meno affidabili (76,5%). Ciò evidenzia maggior difficoltà nell'individuare la causa effettiva del fallimento, compito che richiede un'analisi statica più approfondita, paragonabile alle regole personalizzate utilizzate come ground truth. Al contrario, i *dynamic failure* e le categorie di errore più evidenti, come *crash*, *timeout* e *IPC leak*, mostrano prestazioni significativamente più solide, confermando la maggiore affidabilità del modello quando il fallimento è esplicito e direttamente inferibile dall'analisi del codice o del comportamento del programma. Un ulteriore aspetto interessante riguarda la tendenza del modello a classificare come errato l'output dei programmi anche quando questo è in realtà corretto, come mostrano i risultati relativi alle categorie *static_failure* e *correct*. Questo comportamento potrebbe essere attribuito alla sua natura meno strutturata, che può portare il modello ad interpretare erroneamente alcuni elementi come indicatori di errore, causando così una sovrastima dei fallimenti identificati. In definitiva, emerge un compromesso chiaro: il modello eccelle nell'individuazione di errori manifesti e nell'analisi strutturale del codice, ma

fatica quando è richiesto un livello di astrazione superiore, necessario per interpretare correttamente interazioni concorrenti e proprietà globali del sistema. Si ricorda inoltre che il materiale prodotto nell'ambito di questa tesi è stato pubblicato in un apposito repository GitHub [2].

4.1 Sviluppi futuri

Lo studio condotto offre numerose possibilità di approfondimento ed ampliamento dell'analisi. L'approccio utilizzato si presta infatti ad essere riadattato ed ottimizzato in diversi contesti di valutazione, mantenendo la stessa struttura metodologica oppure evolvendola ulteriormente. In particolare, diverse attività di sviluppo potrebbero contribuire ad estendere e migliorare la qualità dei risultati prodotti, ampliando le possibili applicazioni del framework:

- Estensione dell'analisi mediante LLM all'**intero insieme dei commit**, così da ottenere risultati più completi.
- Ripetizione dell'analisi utilizzando ulteriori **modelli più potenti** o architetture differenti, confrontando le prestazioni ottenute con quelle attuali.
- Utilizzo di ulteriori **strumenti** e regole di analisi statica, confrontando i risultati con quelli ottenuti tramite le **regole custom** e tramite **Semgrep** con regole *Oxdea*.

- **Ottimizzazione** ulteriore dei **prompt**, al fine di ottenere diagnosi ancor più dettagliate ed accurate.
- Conduzione di **analisi personalizzate** per ogni studente, sfruttando la storia dei commit per identificare pattern ricorrenti, difficoltà specifiche e possibili miglioramenti in fase di apprendimento.
- Utilizzo degli LLM per generare **regole statiche personalizzate** per esercizio, evitando la relativa definizione manuale da parte dei professori.
- Sperimentazione dell'impiego degli LLM come **agenti di supporto** agli studenti, in grado di fornire suggerimenti mirati, spiegazioni e feedback personalizzati durante lo sviluppo del codice.

Nel complesso, tali estensioni e miglioramenti consentirebbero non solo di affinare l'accuratezza dell'analisi, ma anche di rendere l'approccio uno strumento più completo, adattivo ed efficace, sia nell'ambito della valutazione automatica sia come supporto alle attività didattiche.

Ringraziamenti

Si conclude con questa tesi il mio percorso di studi, fatto di sacrifici, costanza e impegno, ma anche arricchito dalle numerose conoscenze apprese durante i diversi corsi affrontati. Gli argomenti trattati hanno suscitato fin da subito in me un forte interesse, permettendomi di approfondire e ampliare quanto studiato nel precedente percorso triennale. In particolare, lo sviluppo di progetti ed elaborati mi ha consentito di avvicinarmi allo studio in modo più coinvolgente e stimolante, nonostante il maggiore impegno richiesto. Non sarebbe stato possibile coltivare questo crescente interesse senza la presenza di docenti preparati e capaci di trasmettere passione, curiosità e spunti di riflessione. Ringrazio quindi tutti i professori che mi hanno accompagnato in questo percorso, in particolare i relatori: il Professore *Roberto Natella* ed il Professore *Luigi De Simone*, e i correlatori: gli Ingegneri *Luciano Pianese* e *Carminé Cesarano*, che mi hanno supportato e guidato durante lo svolgimento dell'attività di tesi. Questo traguardo non avrebbe avuto lo stesso significato senza la mia famiglia, sempre pronta a sostenermi e a risollevarmi nei momenti di difficoltà anche

solo con uno sguardo o un sorriso. *Mamma*, il tuo amore, la tua premura e tutti i sacrifici che affronti ogni giorno sono stati la forza che mi ha spinto ad andare avanti. Cercherò sempre di renderti orgogliosa e di ripagare tutto ciò che fai per me. *Carmen*, la tua generosità è stata per me preziosissima e la tua determinazione mi ha ispirato a dare sempre il massimo. *Mery*, grazie per la tua spontaneità, la tua leggerezza e per tutte le risate che sei riuscita a strapparmi anche nei momenti più difficili. Un pensiero speciale va anche a *Vincenzo*, diventato col tempo un fratello acquisito, sempre presente e disponibile in ogni circostanza, anche se con una vita calcisticamente parlando fatta di pochi colori. Non posso non dedicare un pensiero anche alla piccola vecchietta *Lilly*, che con la sua dolcezza e il suo sguardo tenero ha reso meno pesanti le giornate più stressanti, e alla cucciolotta *Cookie*, che con il suo carattere vivace e affettuoso mi è sempre stata accanto rallegrando le mie giornate. Non posso non ringraziare quella che ormai considero una seconda famiglia, fatta di ragazzi dal cuore d'oro e dai valori autentici, che mi hanno regalato momenti di affetto, leggerezza e spensieratezza. Un grazie di cuore va quindi a *Michele*, *Domenico*, *Nicola*, *Pasquale*, *Gloria*, *Davide* e *Noemi*, ai quali auguro di realizzare tutti i propri sogni. Il percorso universitario è senza dubbio più stimolante quando condiviso con persone capaci di comprendere le stesse difficoltà e passioni. Per questo ringrazio in particolare il mio caro amico *Stefano*, compagno di numerosi progetti universitari ma anche

di formazioni al fantacalcio, sempre presente in ogni momento. Ringrazio inoltre *Mario* e *Francesco*, detto “Friccio”, amici e colleghi che mi hanno accompagnato sin dal primo giorno di università, sostenendoci a vicenda durante tutto questo percorso. Infine, ultimo ma non per importanza, ringrazio me stesso per la caparbia, la determinazione e la forza di volontà dimostrate in questi anni, fatti di sacrifici, giornate intere trascorse a studiare e rinunce affrontate con l’obiettivo di raggiungere questo traguardo. Concludo questi pensieri con una promessa a me stesso: non mollare mai davanti a nessun ostacolo e continuare ad affrontare ogni sfida con la stessa tenacia e costanza, senza mai pormi limiti. A tal proposito, vorrei citare una frase del più grande di tutti i tempi:

“È fantastico ripercorrere il passato quando vieni da molto in basso e sai che tutto quel che sei stato, che sei e che sarai non è altro che lotta.”

— *Diego Armando Maradona*

Bibliografia

- [1] Patrick J. Chapman, Cindy Rubio-González, and Aditya V. Thakur. Interleaving static analysis and llm prompting. In *Proceedings of the 13th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*, SOAP 2024, page 9–17, New York, NY, USA, 2024. Association for Computing Machinery.
- [2] Andrea Esposito. Llm-so-exercises. <https://github.com/andreEspo01/LLM-SO-Exercises>, 2026. Online; accessed 11 May 2026.
- [3] Hagit Gabbay and Anat Cohen. Combining llm-generated and test-based feedback in a mooc for programming. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale*, L@S '24, page 177–187, New York, NY, USA, 2024. Association for Computing Machinery.
- [4] Groq. The lpu inference engine. <https://groq.com/>, 2025. Online; accessed 4 March 2025.

- [5] Michael Grottke, Dong Seong Kim, Rajesh Mansharamani, Manoj Nambiar, Roberto Natella, and Kishor S. Trivedi. Recovery from software failures caused by mandelbugs. *IEEE Transactions on Reliability*, 65(1):70–87, 2016.
- [6] Ryan Hecht, Rongxin Liu, Carter Zenke, and David J. Malan. Distributing, collecting, and autograding assignments with github classroom. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 2*, SIGCSE 2023, page 1179, New York, NY, USA, 2023. Association for Computing Machinery.
- [7] Marco Ivaldi. Semgrep rules – c language ruleset. <https://github.com/0xdea/semgrep-rules/tree/main/rules/c>, 2026. Online; accessed 4 May 2026.
- [8] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Zachary Henley, Paul Denny, Michelle Craig, and Tovi Grossman. Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, New York, NY, USA, 2024. Association for Computing Machinery.
- [9] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. Enhancing static analysis for practical bug detection: An llm-integrated approach. *Proc. ACM Program. Lang.*, 8(OOPSLA1), April 2024.

- [10] Wenhan Lyu, Yimeng Wang, Tingting (Rachel) Chung, Yifan Sun, and Yixuan Zhang. Evaluating the effectiveness of llms in introductory computer science education: A semester-long field study. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale, L@S '24*, page 63–74, New York, NY, USA, 2024. Association for Computing Machinery.
- [11] Magica. Gpt-4o vs gpt-oss-20b. https://magica.com/blog/compare/gpt-4o-vs-gpt-oss-20b?utm_source=chatgpt.com, 2026. Online; accessed 19 May 2026.
- [12] Microsoft Azure. Azure portal. <https://portal.azure.com>, 2026. Online; accessed 13 April 2026.
- [13] Mohammad Mahdi Mohajer, Reem Aleithan, Nima Shiri Harzevili, Moshi Wei, Alvine Boaye Belle, Hung Viet Pham, and Song Wang. Effectiveness of chatgpt for static analysis: How far are we? In *Proceedings of the 1st ACM International Conference on AI-Powered Software*, AIware 2024, page 151–160, New York, NY, USA, 2024. Association for Computing Machinery.
- [14] Roberto Natella. Evaluation of systems programming exercises through tailored static analysis. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V*.

1, SIGCSETS 2025, page 826–832, New York, NY, USA, 2025.
Association for Computing Machinery.

[15] Ollama. Run llms locally. <https://ollama.com/>, 2026.
Online; accessed 25 February 2026.

[16] Prompting Guide. React: Reason + act. <https://www.promptingguide.ai/techniques/react>, 2026. Online;
accessed 25 March 2026.